Honeypot en AWS: monitorización y análisis de actividad real con Wazuh

Marta Bargués Bosch



1.Introducción

Este proyecto se centra en la monitorización y el análisis de la actividad recibida por un honeypot expuesto a Internet. A partir de los registros recogidos, se estudiaron los intentos de acceso, las peticiones web y las acciones realizadas dentro del entorno señuelo.

El laboratorio se había desplegado previamente en AWS y contaba con dos servicios: Cowrie, que simulaba un servidor SSH, y Nginx, que publicaba la web ficticia de Aegis Bank. Los registros de ambos servicios se centralizaron en un servidor Wazuh instalado en una máquina virtual local. La comunicación entre los dos entornos se realizó mediante una VPN WireGuard.

Esta memoria recoge la configuración necesaria para consultar esos registros en Wazuh y el análisis posterior de la actividad observada. Se revisaron intentos de autenticación, comandos introducidos, búsquedas de recursos web y transferencias de archivos al honeypot.

El trabajo se planteó desde una perspectiva Blue Team, utilizando los eventos para investigar comportamientos sospechosos y reconstruir algunas de las sesiones registradas. Las capturas incluidas muestran las evidencias utilizadas para explicar los hallazgos y sus limitaciones.

2.Objetivos

El objetivo principal de esta parte del proyecto fue analizar los ataques recibidos por el honeypot utilizando Wazuh, para entender qué intentaban hacer los atacantes y qué información podíamos obtener de los registros.

Para ello, nos planteamos los siguientes objetivos:

- Configurar el envío de los registros de Cowrie y Nginx a Wazuh y comprobar que se recibían correctamente.
- Crear una regla personalizada para poder visualizar y analizar los eventos de Cowrie.
- Analizar los intentos de acceso por SSH, las contraseñas utilizadas y los comandos ejecutados dentro del honeypot.
- Revisar las peticiones web para identificar escaneos, búsquedas de archivos y posibles intentos de explotación.
- Relacionar los eventos para reconstruir algunas de las sesiones de los atacantes.

- Analizar los archivos capturados y comprobar sus hashes en VirusTotal, sin ejecutar las muestras.
- Crear un dashboard en Wazuh para visualizar los ataques, las IP con mayor actividad y su procedencia geográfica aproximada.

Con este análisis queríamos poner en práctica el trabajo de un analista Blue Team, aprendiendo a investigar los eventos y a distinguir entre lo que realmente mostraban los registros y lo que solo podíamos sospechar.

3.Descripción del laboratorio

Para realizar el análisis se utilizó el laboratorio desplegado en la primera parte del proyecto. La infraestructura combinaba un honeypot alojado en AWS y un servidor Wazuh instalado en una máquina virtual local.

3.1 Servicios del honeypot

La instancia Honeypot-Aegis utilizaba Ubuntu Server y contaba con dos servicios expuestos a Internet:

- **Cowrie:** simulaba un servidor SSH y registraba las conexiones, los usuarios y contraseñas utilizados, los comandos introducidos y las transferencias de archivos.
- **Nginx**: publicaba la página del banco ficticio Aegis Bank. Sus registros permitían consultar las IP de origen, las rutas solicitadas, los métodos HTTP y las respuestas del servidor.

Las conexiones al puerto 22 se redirigían al puerto 2222, donde escuchaba Cowrie. El SSH administrativo utilizaba el puerto 22222 y se accedía a él mediante la VPN. De esta forma, las sesiones del servicio señuelo quedaban separadas del acceso real al servidor.

3.2 Comunicación con Wazuh

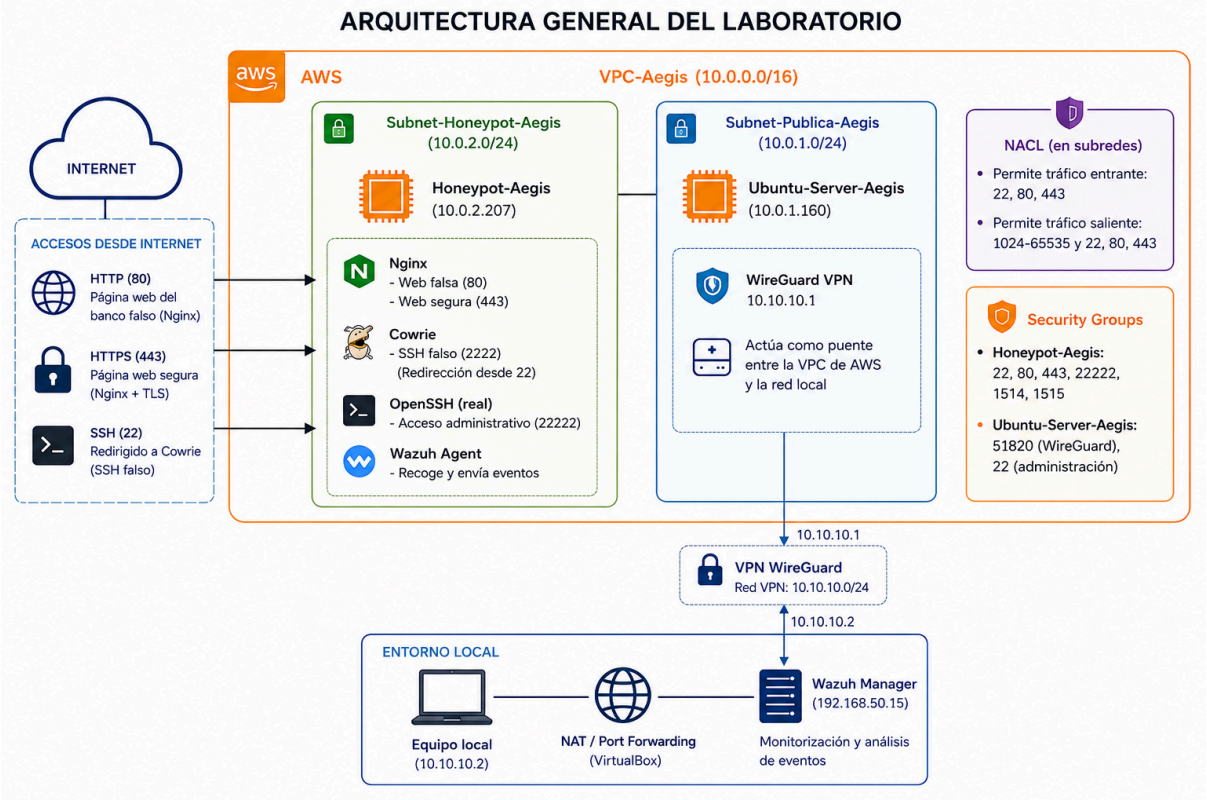
En el honeypot se había instalado un agente de Wazuh para enviar los registros al servidor de monitorización. La comunicación se realizaba mediante WireGuard, utilizando el servidor Ubuntu-Server-Aegis como intermediario entre AWS y el entorno local.
Wazuh se encontraba en una máquina virtual independiente e incluía Manager, Indexer y Dashboard. Desde este entorno se procesaron los registros, se consultaron las alertas y se crearon las visualizaciones utilizadas durante el análisis.

Esta distribución permitió mantener el servidor de monitorización fuera de la exposición directa a Internet, mientras el honeypot permanecía accesible para recibir conexiones.

3.3 Alcance del análisis

El análisis se centró en la actividad registrada por Cowrie y Nginx durante los periodos de monitorización. Se estudiaron tanto los intentos de acceso como las acciones posteriores dentro del entorno simulado.

En Cowrie, los accesos aceptados y los comandos registrados correspondían al servicio señuelo. Por tanto, estos eventos no demostraban que se hubiera accedido al sistema real ni que los programas transferidos se hubieran ejecutado en él.
Las pruebas realizadas durante el despliegue se tuvieron en cuenta para diferenciarlas de las conexiones externas. Además, la geolocalización de las IP se utilizó únicamente como una referencia aproximada de su procedencia.



4. Recogida e integración de registros en Wazuh

Una vez conectado el agente Honeypot-Aegis con Wazuh Manager, configuramos la recogida de los registros de Cowrie y Nginx para poder consultar toda la actividad desde Wazuh.

4.1 Configuración de los registros de Cowrie

Primero revisamos dónde guardaba Cowrie sus registros y encontramos dos archivos dentro de /home/cowrie/cowrie/var/log/cowrie/:

cowrie.log: contiene información sobre el funcionamiento de Cowrie y la actividad registrada.
cowrie.json: guarda los eventos en formato JSON, con información como la IP de origen, los intentos de acceso, las contraseñas utilizadas y los comandos introducidos.

```
ubuntu@honeypot-aegis:~$ sudo find / -type f \( -name "cowrie.json" -o -name "cowrie.log" \) 2>/dev/null
/home/cowrie/cowrie/var/log/cowrie/cowrie.log
/home/cowrie/cowrie/var/log/cowrie/cowrie.json
```

Decidimos utilizar cowrie.json, ya que al tener los datos separados por campos nos resultaría más sencillo analizarlos después en Wazuh.

Así que, editamos la configuración del agente en /var/ossec/etc/ossec.conf y añadimos la ruta del archivo.

```
ubuntu@honeypot-aegis:~$ sudo grep -i cowrie /var/ossec/logs/ossec.log | tail -20
2026/08/14 11:24:45 wazuh-logcollector: INFO: (1950): Analyzing file: '/home/cowrie/cowrie/var/log/cowrie/cowrie.json'.
```

4.2 Comprobación de los eventos de Cowrie

Una vez configurada la recogida de registros, revisamos cowrie.json para ver qué actividad había registrado el honeypot.

Encontramos una conexión SSH desde la IP pública 186.158.200.26 al puerto 2222 de Cowrie. El registro mostraba la IP de origen, la fecha y un identificador de sesión que nos permitiría relacionar los diferentes eventos de esa conexión.

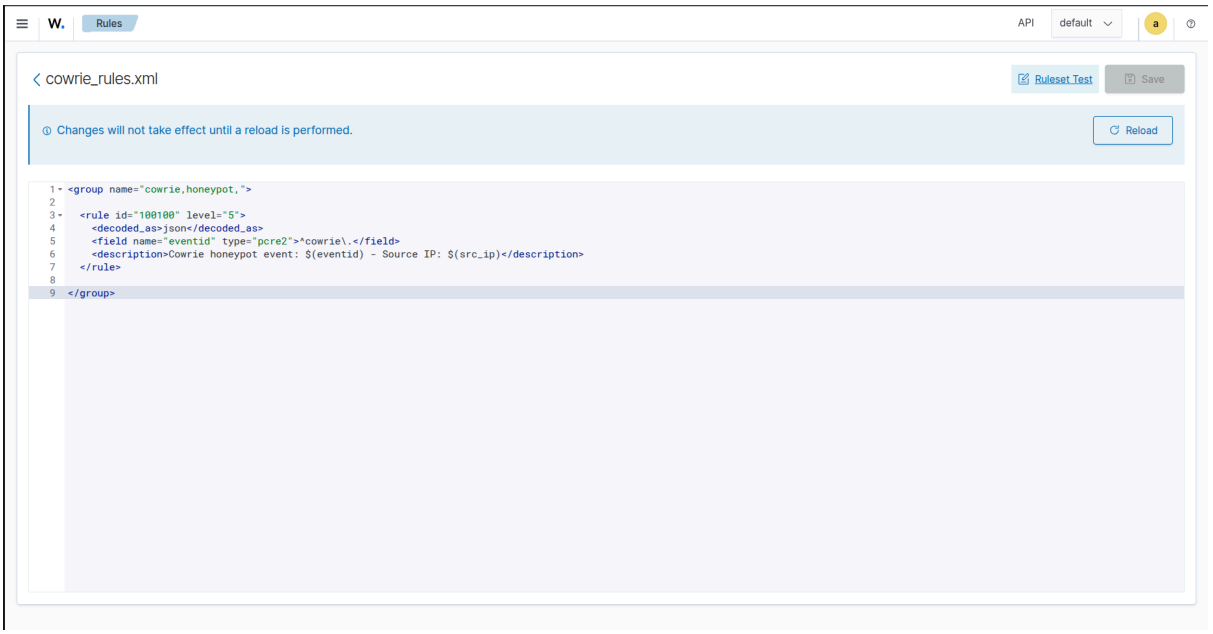
El evento aparecía como cowrie.session.closed, lo que indicaba que la sesión se había cerrado, pero no nos permitía saber si el atacante había conseguido autenticarse o ejecutar algún comando.

```
ubuntu@honeypot-aegis:~$ sudo tail -n 1 /home/cowrie/cowrie/var/log/cowrie/cowrie.json
{"session":"772c24d724d9","protocol":"ssh","src_ip":"186.158.200.26","src_port":2087,"dst_ip":"10.0.2.207","dst_port":2222,"duration_ms":0,"eventid":
owrie.session.closed","sensor":"honeypot-aegis","uuid":"3322f1ce-94ae-11f1-9f5e-0addca92d8af","timestamp":"2026-08-14T11:32:56.924564Z","message":"Co
ection lost after 0 milliseconds"}
ubuntu@honeypot-aegis:~$
```

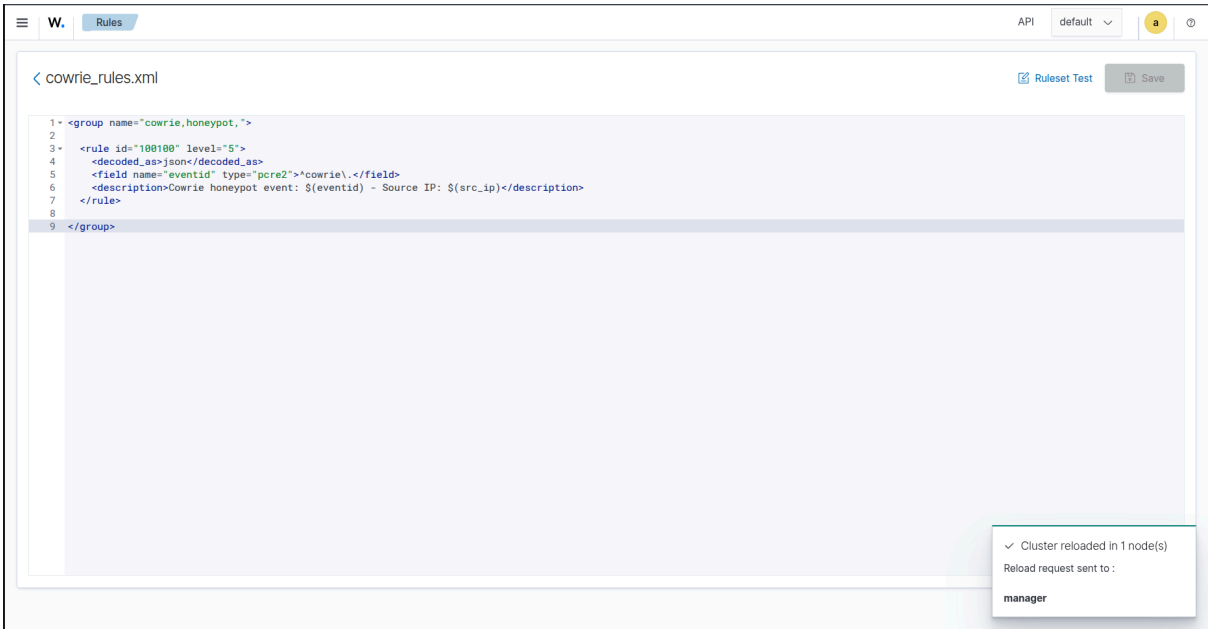
Aunque el agente ya estaba leyendo los registros, todavía no podíamos verlos como alertas en Threat Hunting. Por eso, el siguiente paso fue crear una regla personalizada en Wazuh para poder identificarlos y analizarlos más fácilmente.

4.3 Creación de una regla personalizada para Cowrie

Para poder ver los eventos de Cowrie como alertas en Wazuh, creamos una regla personalizada en el archivo /var/ossec/etc/rules/cowrie_rules.xml.
Le asignamos el identificador 100100 y un nivel de alerta 5. La regla buscaba los eventos en formato JSON cuyo campo eventid empezara por cowrie..



También configuramos la descripción para que mostrara el tipo de evento y la IP de origen. Así podíamos identificar mejor si se trataba de una conexión, un intento de acceso o un comando introducido en el honeypot.
Esta regla permitió visualizar la actividad de Cowrie en Wazuh, aunque todos los eventos recibieran el mismo nivel de alerta, independientemente de su gravedad.
Después de guardar los cambios, recargamos las reglas desde el Dashboard y comprobamos que Wazuh había aplicado la configuración.

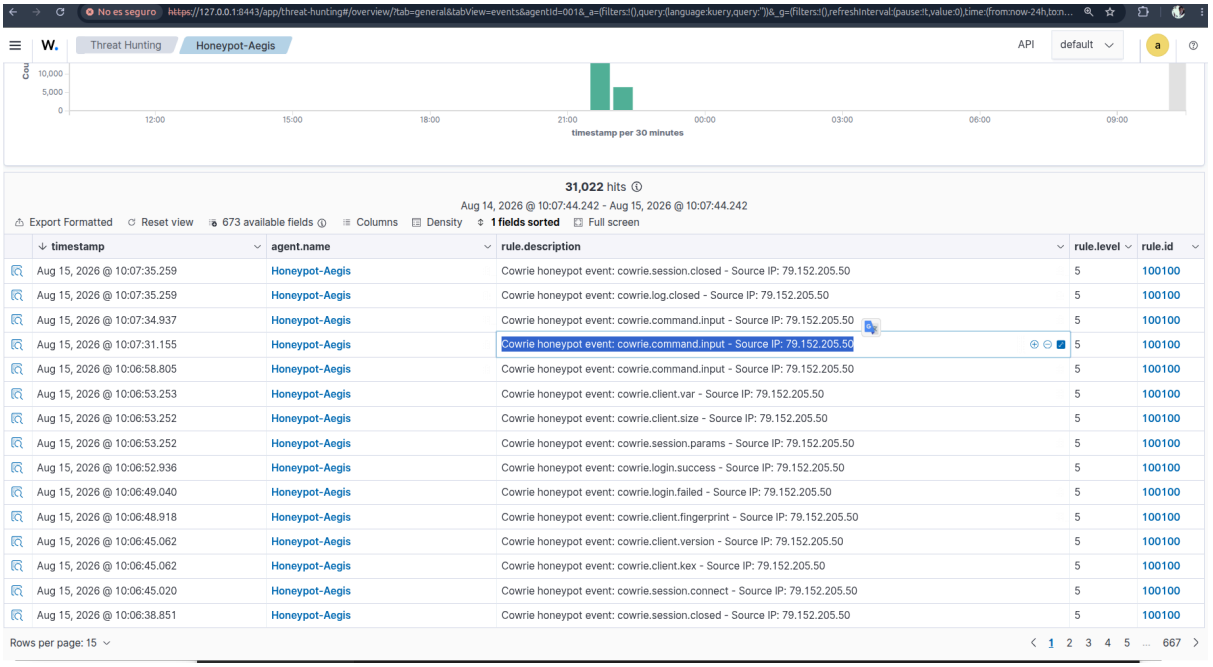

Rules
API
default
a
cowrie_rules.xml
Ruleset Test
Save
1 <group name="cowrie,honeypot,">
2
3 <rule id="100100" level="5">
4 <decoded_as>json</decoded_as>
5 <field name="eventid" type="pcre2">^cowrie\.</field>
6 <description>Cowrie honeypot event: $(eventid) - Source IP: $(src_ip)</description>
7 </rule>
8
9 </group>
Cluster reloaded in 1 node(s)
Reload request sent to :
manager

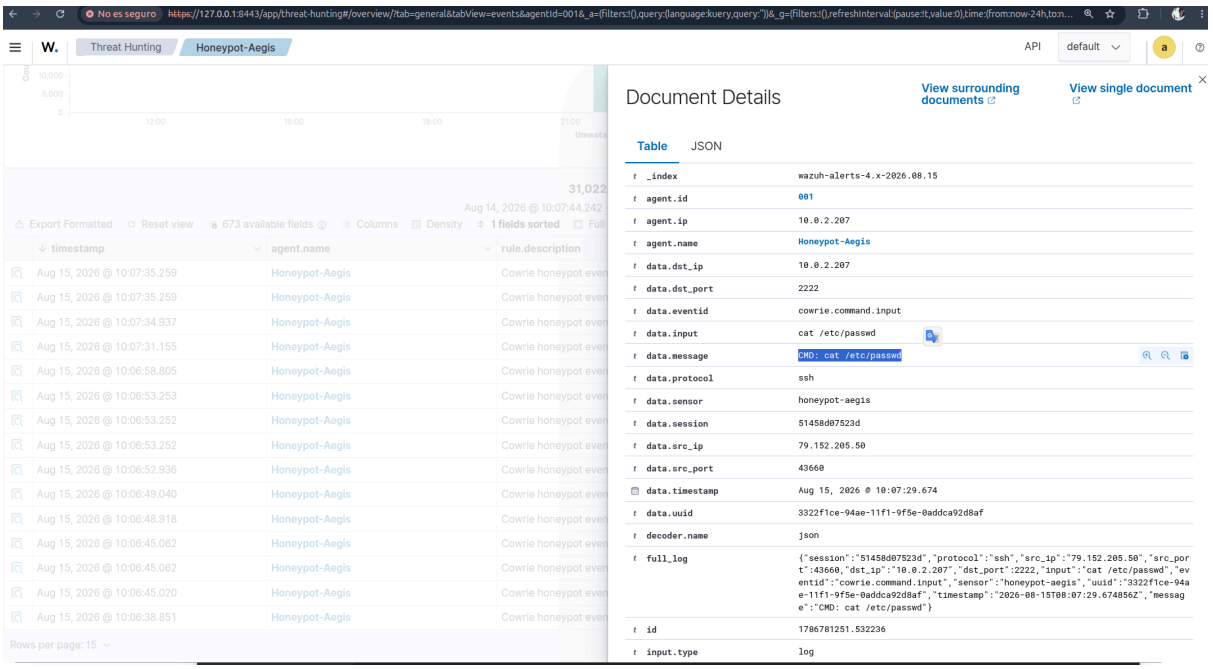
4.4 Validación y visualización de los eventos en Wazuh

Una vez creada la regla, utilizamos wazuh-logtest para comprobar que Wazuh reconocía los eventos de Cowrie y les asignaba la regla 100100.
Después realizamos una conexión SSH de prueba contra el honeypot y revisamos los resultados en Threat Hunting.
Comprobamos que los eventos aparecían asociados al agente Honeypot-Aegis, con el nivel de alerta 5 que habíamos configurado.
Entre los registros encontramos la conexión inicial, intentos de autenticación, comandos introducidos y el cierre de la sesión.
Al revisar uno de los eventos vimos que se había registrado el comando:

cat /etc/passwd

Este aparecía identificado como cowrie.command.input, junto con la IP de origen y el identificador de la sesión.

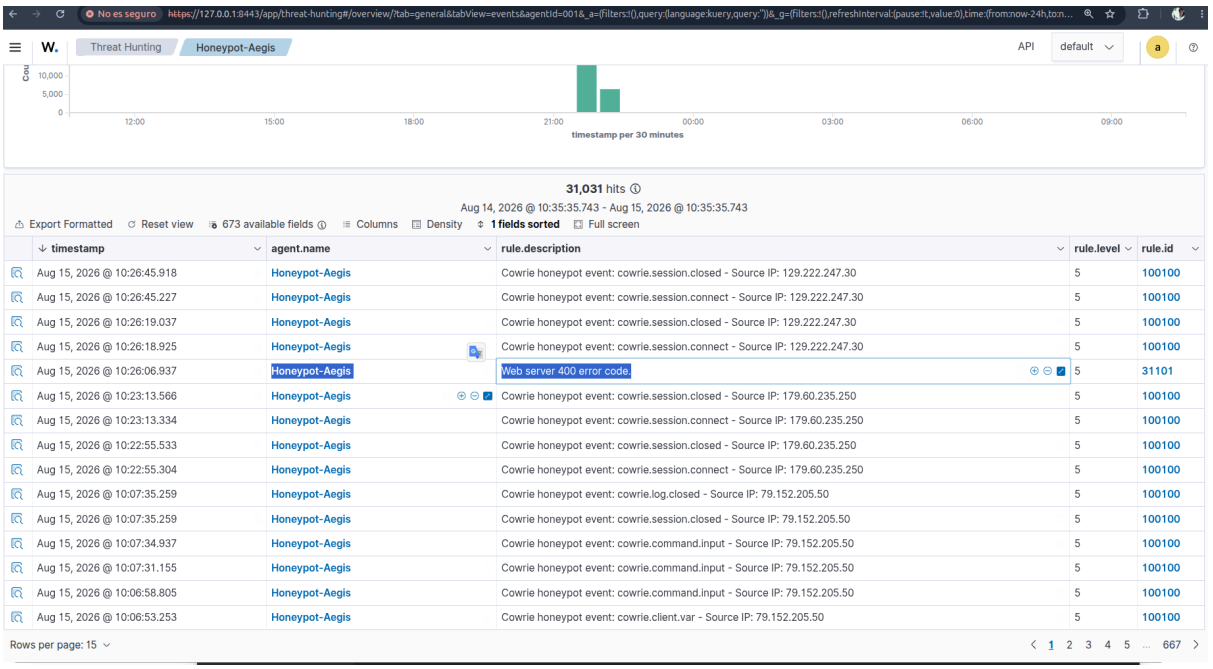




Con estas pruebas comprobamos que Cowrie estaba enviando los registros a Wazuh y que podíamos consultar la actividad desde el Dashboard.
Una vez comprobado su funcionamiento, ya podíamos empezar a analizar las conexiones reales que había recibido el honeypot.

4.5 Monitorización de los accesos al servidor web

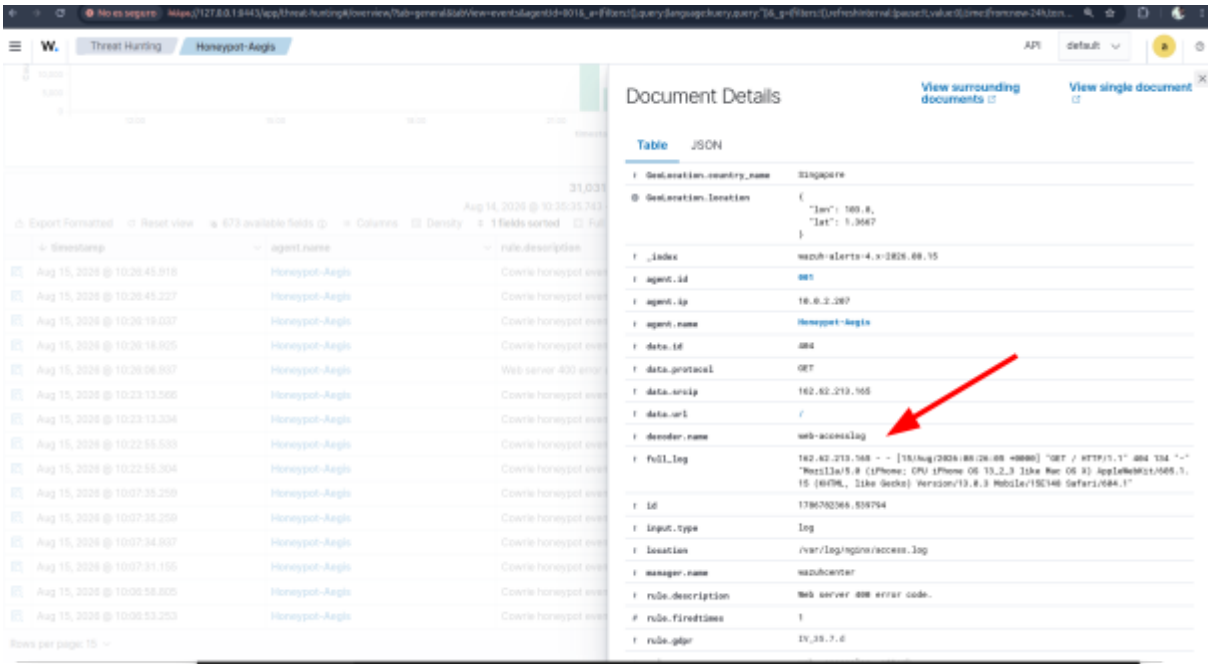
Además de los registros de Cowrie, configuramos el agente de Wazuh para recoger la actividad del servidor web Nginx, que guardaba las peticiones recibidas en /var/log/nginx/access.log.
Desde Threat Hunting comprobamos que los eventos aparecían asociados al agente Honeypot-Aegis. Al revisar las alertas podíamos ver la IP de origen, la página solicitada y la respuesta del servidor.



En uno de los registros encontramos una petición desde la IP 162.62.213.165:

GET / HTTP/1.1 ⟶ 404

Wazuh había identificado el evento mediante el decodificador web-accesslog y generado una alerta con la regla 31101.
Aunque la descripción indicaba Web server 400 error code, al revisar el registro vimos que el código real era 404, es decir, que el servidor no había encontrado el recurso solicitado.



Con esta comprobación vimos que Wazuh también estaba recogiendo los registros de Nginx y que podíamos analizar las peticiones recibidas por la web.

5.Análisis de la actividad SSH y web

Una vez comprobamos que Wazuh recibía los registros de Cowrie y Nginx, empezamos a analizar la actividad que había llegado desde Internet.
Desde Threat Hunting buscamos eventos por IP, tipo de actividad e identificador de sesión para intentar entender qué hacían los equipos que se conectaban al honeypot.

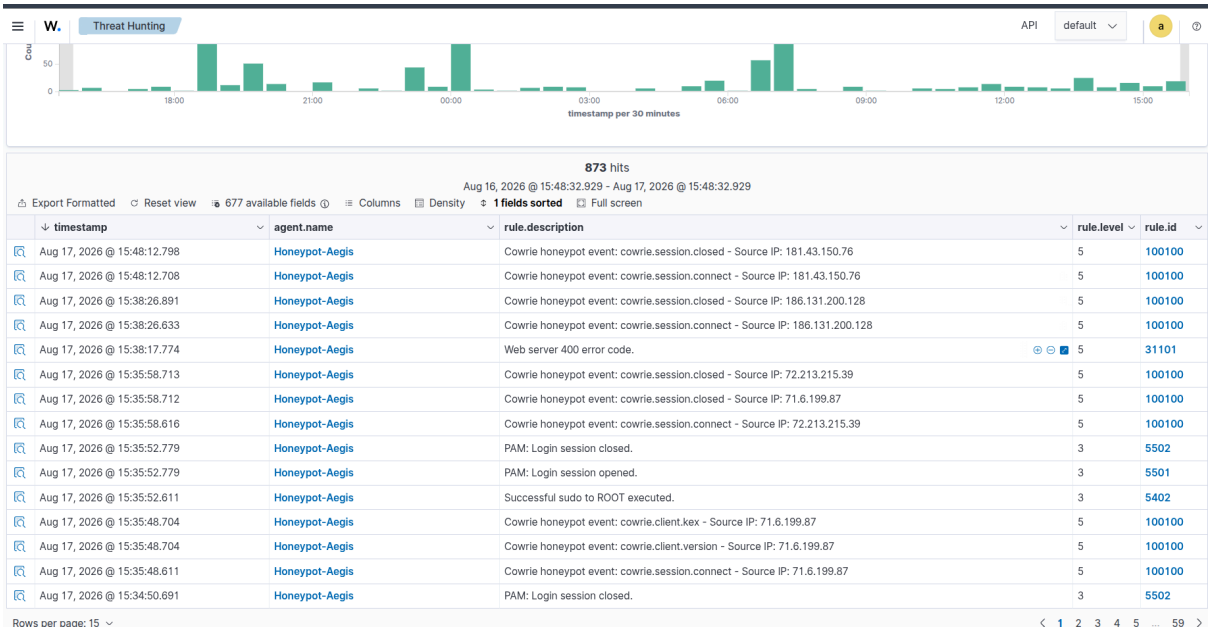
5.1 Análisis de la actividad SSH registrada por Cowrie

Al revisar los primeros registros encontramos conexiones desde diferentes direcciones IP. Algunas se cerraban casi inmediatamente, sin que aparecieran intentos de autenticación. Esto podía indicar que se trataba de escaneos automatizados buscando servidores SSH, aunque no podíamos asegurarlo únicamente por la duración de la conexión.

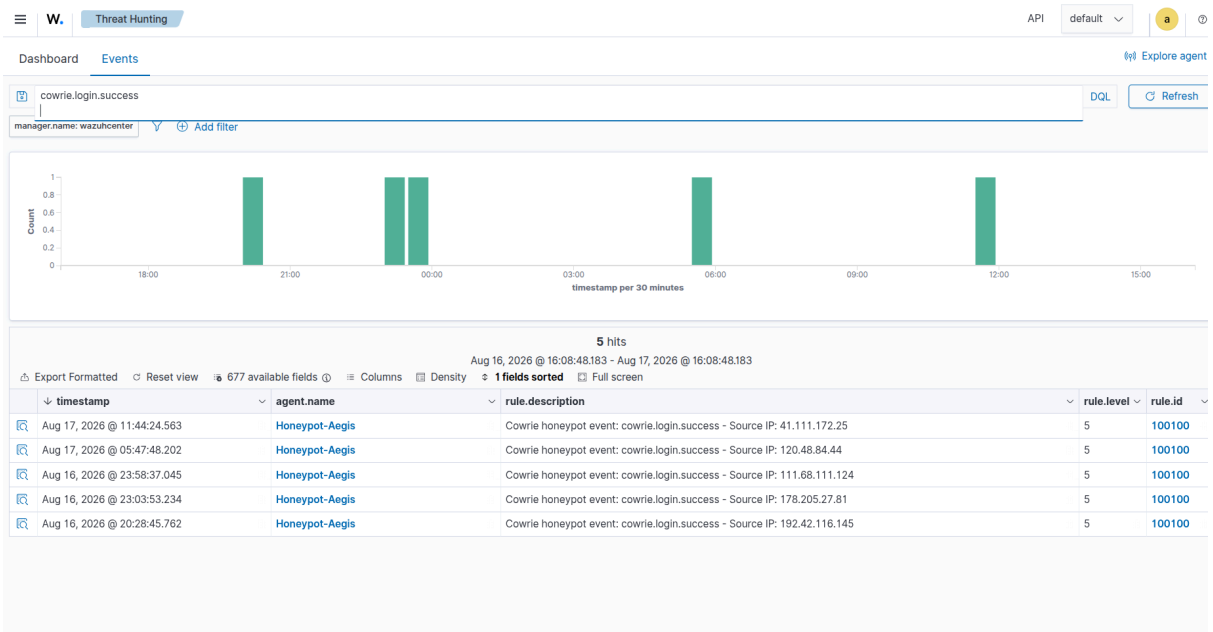
Para analizar mejor la actividad, nos fijamos en los siguientes eventos:

- **cowrie.session.connect** - Se establece una conexión con Cowrie.
- **cowrie.session.closed** - Se cierra la conexión
- **cowrie.login.failed** - Se intenta acceder con unas credenciales que Cowrie rechaza.
- **cowrie.login.success** - Cowrie acepta las credenciales y permite entrar en el entorno simulado.
- **cowrie.command.input** - Se introduce un comando dentro del honeypot.

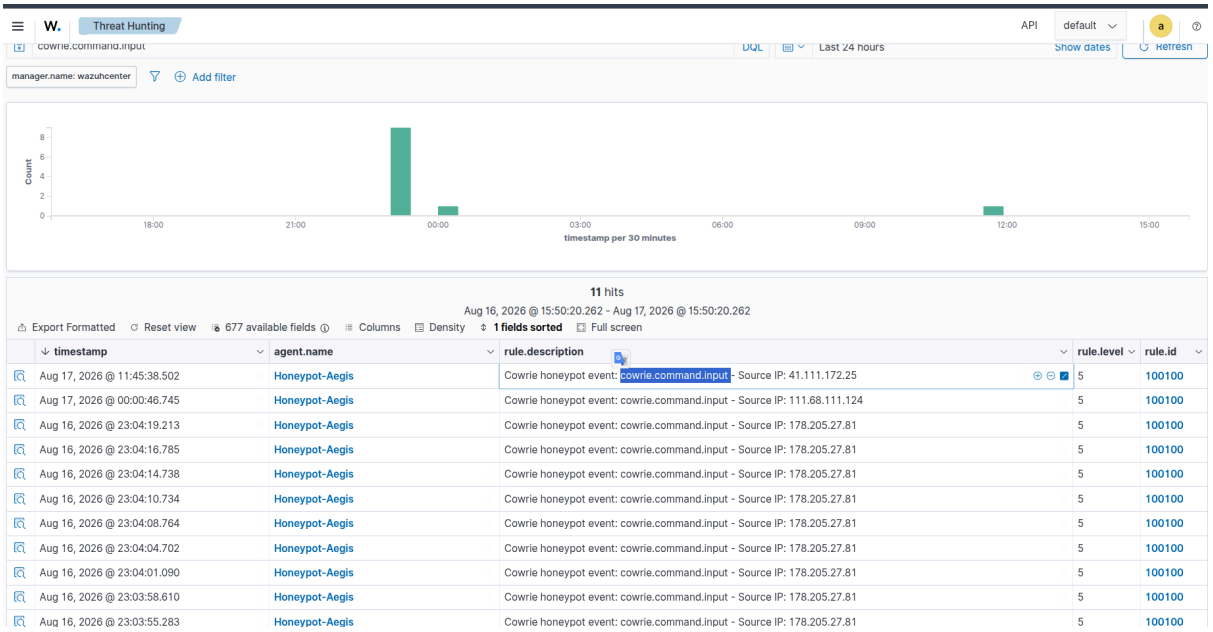
Primero buscamos los eventos cowrie.login.success y encontramos cinco accesos aceptados desde cinco IP diferentes.



Estos accesos no significaban que hubieran comprometido un servidor real, sino que Cowrie había aceptado las credenciales para permitirles interactuar con el entorno simulado.



Después buscamos los eventos cowrie.command.input y encontramos 11 comandos registrados. Varios procedían de la IP 178.205.27.81 y se habían introducido en pocos segundos.

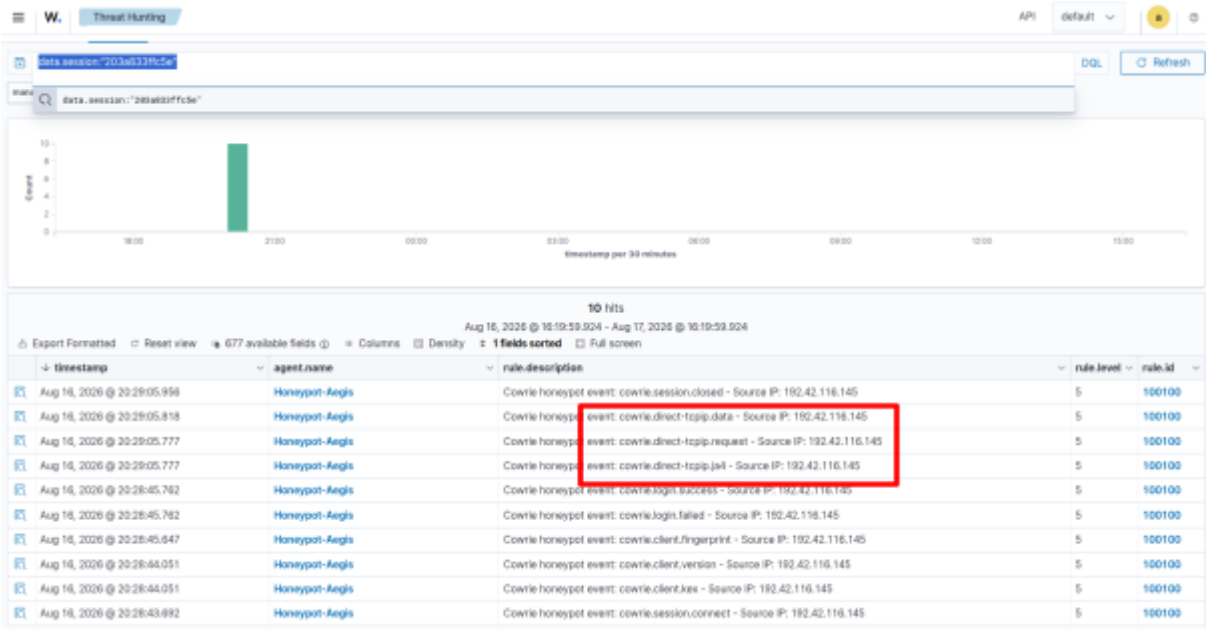


timestamp	agent.name	rule.description	rule.level	rule.id
Aug 17, 2026 @ 11:45:38.502	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 41.111.172.25	5	100100
Aug 17, 2026 @ 00:00:46.745	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 111.68.111.124	5	100100
Aug 16, 2026 @ 23:04:19.213	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:16.785	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:14.738	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:10.734	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:08.764	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:04.702	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:04:01.090	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:03:58.610	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100
Aug 16, 2026 @ 23:03:55.283	Honeypot-Aegis	Cowrie honeypot event: cowrie.command.input - Source IP: 178.205.27.81	5	100100

Como las búsquedas se habían realizado en intervalos de tiempo ligeramente diferentes, no utilizamos estos resultados para calcular porcentajes, sino para localizar las sesiones que nos interesaba investigar.
A partir de aquí seleccionamos las conexiones en las que se habían introducido comandos para reconstruir qué había ocurrido dentro del honeypot después del acceso.

5.2 Análisis de las sesiones y comandos introducidos

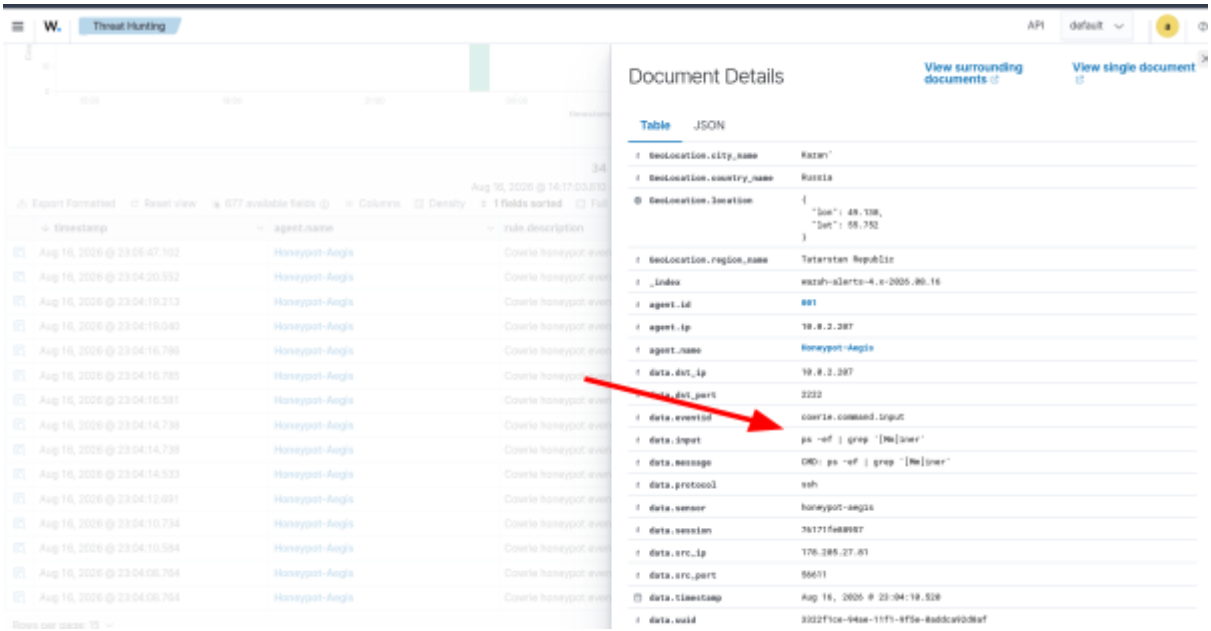
Después de revisar los accesos, quisimos investigar qué hacían los atacantes una vez dentro del honeypot.
Por eso, utilizamos el campo data.session, que nos permitía agrupar los eventos de una misma conexión y seguir las acciones.
Primero analizamos la sesión 203a833ffc5e, procedente de la IP 192.42.116.145.
Encontramos 10 eventos, entre ellos un intento de acceso fallido, otro aceptado y el cierre de la conexión.
También aparecieron eventos direct-tcpip, relacionados con solicitudes de reenvío de conexiones TCP a través de SSH. Esto nos llamó la atención porque mostraba que no toda la actividad SSH consistía en ejecutar comandos.



Después revisamos otra sesión, procedente de la IP 178.205.27.81, donde encontramos el siguiente comando:

ps -ef | grep '[Mm]iner'

Este comando buscaba procesos que contuvieran la palabra miner, lo que podría ser que el atacante estaba comprobando si había algún minero de criptomonedas ejecutándose en el sistema.



Así y todo, con este registro no podíamos confirmar que realmente existiera un minero ni cuál era la intención del atacante.

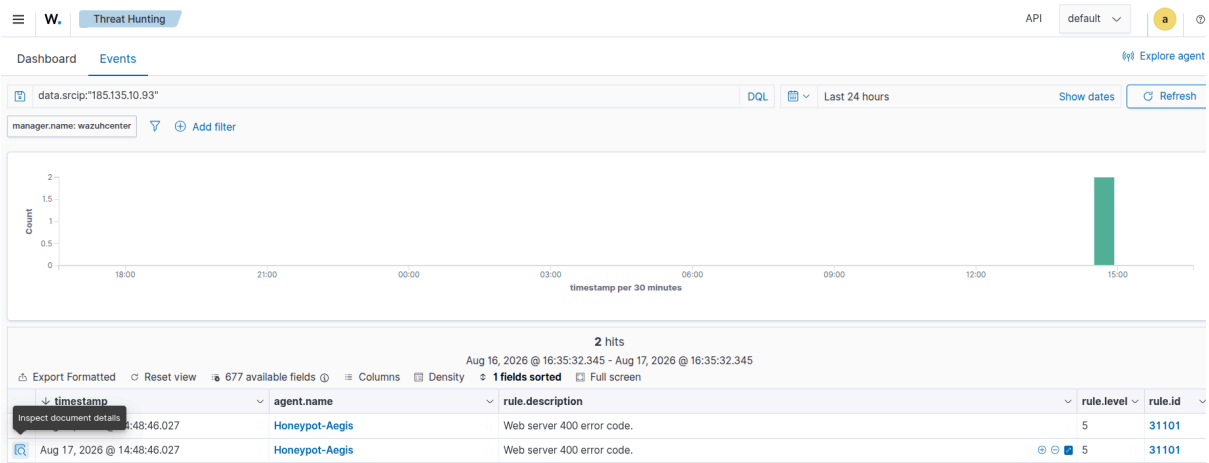
También observamos que varios comandos de esta IP se habían ejecutado con pocos segundos de diferencia, algo que podía indicar el uso de algún script automatizado.
Con este análisis conseguimos reconstruir parte de la actividad de las sesiones y localizar comportamientos que merecían una revisión más detallada, especialmente los relacionados con la descarga o ejecución de archivos.

5.3 Análisis de la actividad contra el servidor web

Después de analizar las conexiones SSH, pasamos a revisar las peticiones recibidas por Nginx.
Para localizar los registros utilizamos el siguiente filtro en Threat Hunting:

location:"/var/log/nginx/access.log"

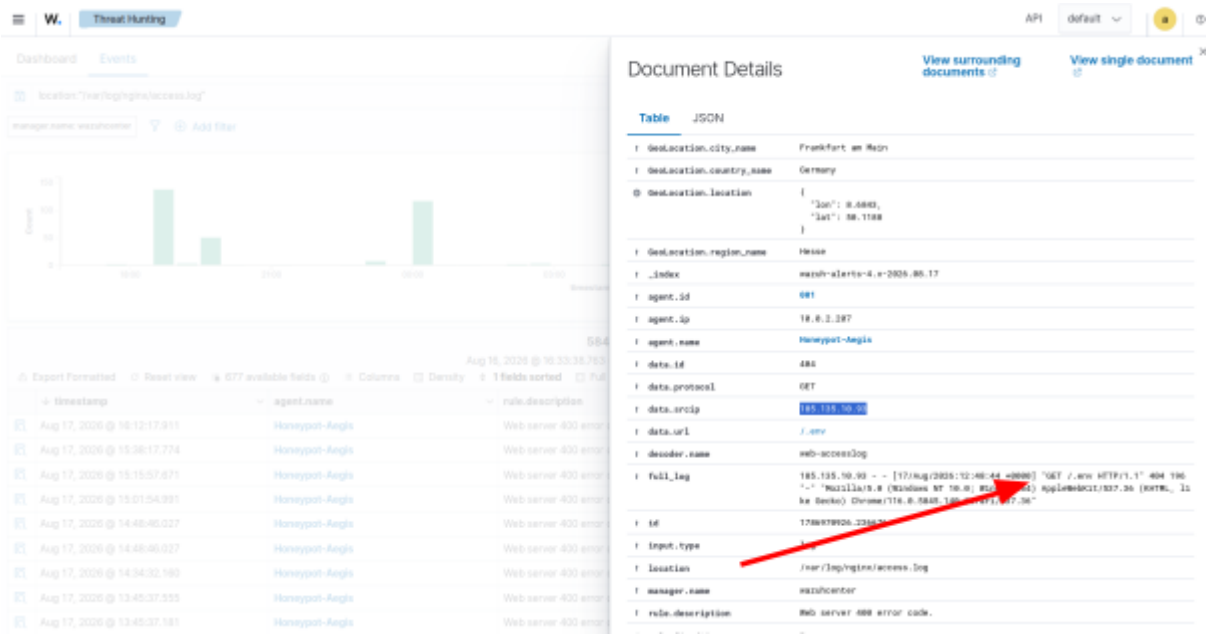
Revisamos las IP de origen, las páginas solicitadas, los códigos de respuesta y la cantidad de peticiones realizadas.
Uno de los casos que encontramos correspondía a la IP 185.135.10.93, desde la que se habían registrado dos eventos durante el intervalo consultado.



Al revisar uno de ellos vimos que se había intentado acceder al siguiente archivo:

GET /.env HTTP/1.1

El servidor respondió con un código 404, indicando que no había encontrado el archivo solicitado.



Esta petición nos llamó la atención porque los archivos .env suelen guardar información de configuración de las aplicaciones y, en ocasiones, pueden contener contraseñas, credenciales de bases de datos o claves de acceso a servicios.
Por eso, buscar este tipo de archivos es una técnica habitual durante el reconocimiento de servidores web.
En nuestro caso, la respuesta 404 indicaba que el archivo no se había encontrado.
Aunque la petición era sospechosa, dos eventos no eran suficientes para asegurar que se tratara de un escaneo automatizado.
A partir de aquí continuamos revisando otras direcciones IP que habían realizado numerosas peticiones en poco tiempo, buscando posibles patrones de reconocimiento o intentos de explotación.

5.4 Geolocalización de las direcciones IP

Durante el análisis observamos que los eventos de Cowrie mostraban las direcciones IP de origen, pero no su ubicación geográfica.
Como queríamos representar los ataques en un mapa, revisamos la configuración de Wazuh para añadir esta información.

Primero habilitamos la opción analysisd.geoip_jsonout=1 en Wazuh. Después revisamos el procesamiento de los registros en Filebeat, donde se utilizaba geoip para obtener información geográfica a partir de las IP.

```
# DO NOT TOUCH THIS FILE. The default configuration
# is at ossec.conf. More information at:
# https://documentation.wazuh.com
#
# This file should be handled with care. It contain
# run time modifications that can affect the use
# of ossec. Only change it if you know what you
# are doing. Again, look first at ossec.conf
# for most of the things you want to change.


# Analysisd default rule timeframe.
analysisd.default_timeframe=360
# Analysisd stats maximum diff.
analysisd.stats_maxdiff=999000
# Analysisd stats minimum diff.
analysisd.stats_mindiff=1250
# Analysisd stats percentage (how much to differ from average)
analysisd.stats_percent_diff=150
# Analysisd FTS list size.
analysisd.fts_list_size=32
# Analysisd FTS minimum string size.
analysisd.fts_min_size_for_str=14
# Analysisd Enable the firewall log (at logs/firewall/firewall.log)
# 1 to enable, 0 to disable.
analysisd.log_fw=1
# Maximum number of fields in a decoder (order tag) [32..1024]
analysisd.decoder_order_size=256
# Output GeoIP data at JSON alerts
analysisd.geoip_jsonout=1
# Maximum label cache age (margin seconds with no reloading) [0..60]
File to insert [from ./]:
^G Help                    M-F New Buffer             ^X Execute Command
^C Cancel                  M-N No Conversion          ^T Browse
```

Antes de realizar cambios, hicimos una copia de seguridad del archivo:

```
user@wazuhcenter:~$ sudo cp /usr/share/filebeat/module/wazuh/alerts/ingest/pipeline.json \
/usr/share/filebeat/module/wazuh/alerts/ingest/pipeline.json.bak
user@wazuhcenter:~$ sudo nano /usr/share/filebeat/module/wazuh/alerts/ingest/pipeline.json
```

Modificamos la configuración para que también se procesaran las direcciones IP de los eventos de Cowrie.

```
 GNU nano 7.2                                /usr/share/filebeat/module/wazuh/alerts/ingest/pipeline.json
{
  "description": "Wazuh alerts pipeline",
  "processors": [
    { "json" : { "field" : "message", "add_to_root": true } },
    {
      "set": {
        "field": "data.aws.region",
        "value": "{{data.aws.awsRegion}}",
        "override": false,
        "ignore_failure": true,
        "ignore_empty_value" : true
      }
    },
    {
      "set": {
        "field": "data.aws.accountId",
        "value": "{{data.aws.aws_account_id}}",
        "override": false,
        "ignore_failure": true,
        "ignore_empty_value" : true
      }
    },
    {
      "geoip": {
        "field": "data.srcip",
        "target_field": "GeoLocation",
        "properties": ["city_name", "country_name", "region_name", "location"],
        "ignore_missing": true,
        "ignore_failure": true
      }
    },
    {
      "geoip": {
```

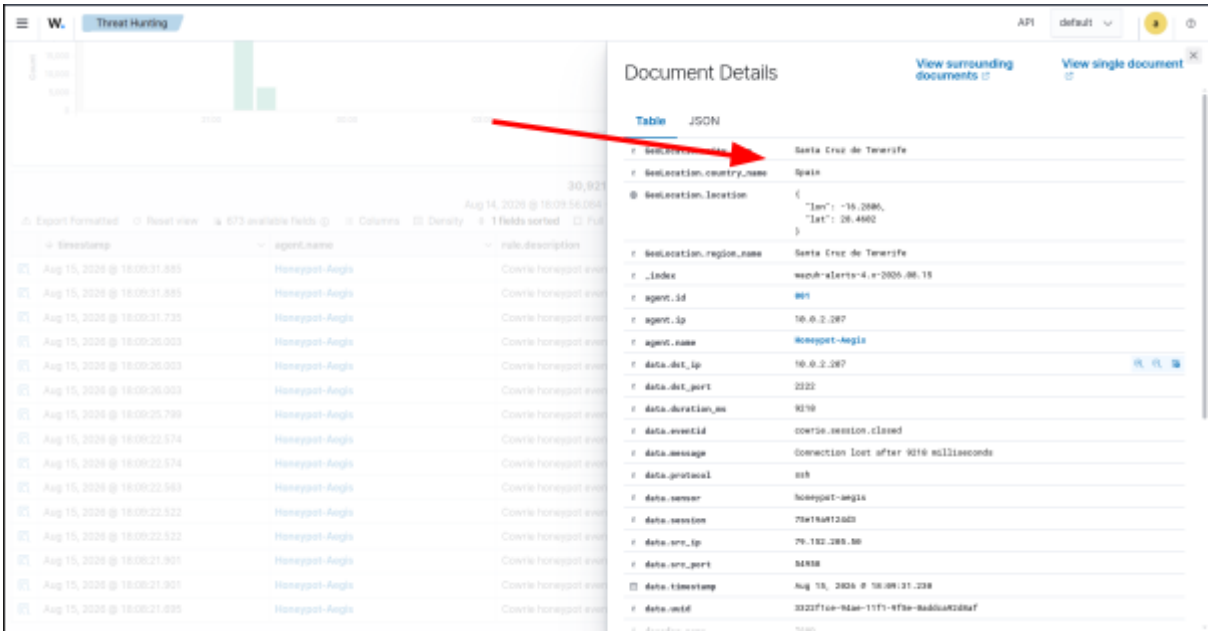
Finalmente, volvimos a cargar los pipelines y reiniciamos Filebeat para aplicar los cambios.

```
user@wazuhcenter:~$ python3 -m json.tool /usr/share/filebeat/module/wazuh/alerts/ingest/pipeline.json >/dev/null
user@wazuhcenter:~$ sudo filebeat test config
Config OK
```

```
user@wazuhcenter:~$ sudo filebeat setup --pipelines
Loaded Ingest pipelines
user@wazuhcenter:~$ sudo systemctl restart filebeat
```

Al consultar los nuevos eventos en Threat Hunting, comprobamos que ya aparecían los campos de geolocalización

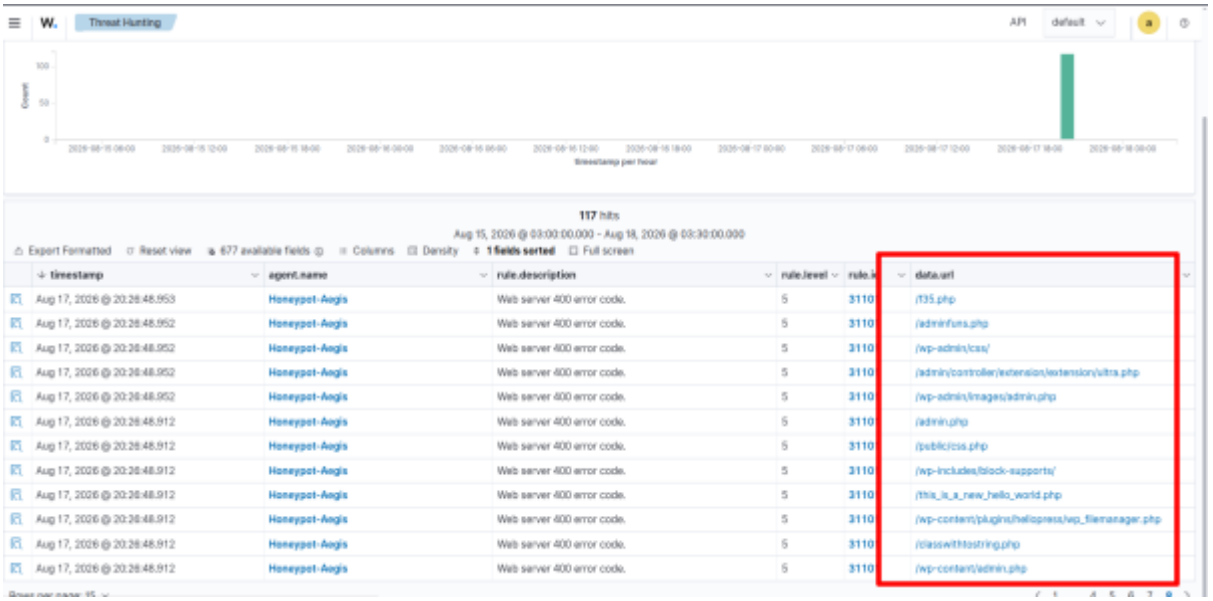
Con esta información ya podíamos visualizar la procedencia geográfica aproximada de las conexiones y utilizarla posteriormente en nuestro dashboard.



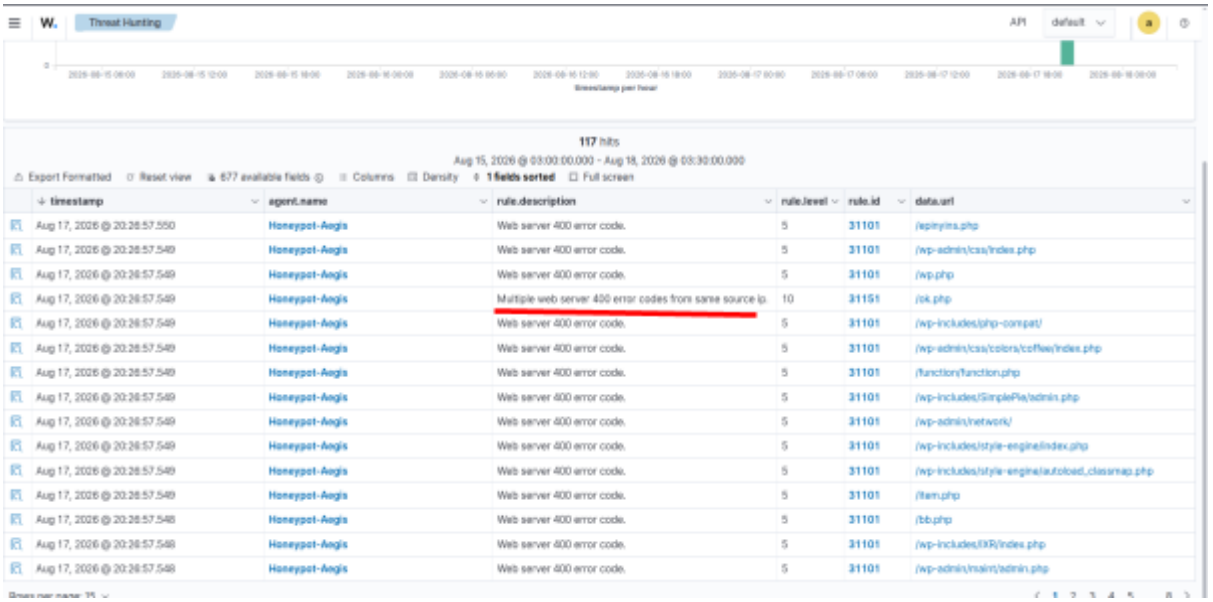
Sin embargo, hay que tener en cuenta que la ubicación de una IP no indica necesariamente dónde se encuentra el atacante, ya que podría estar utilizando una VPN, un proxy o un servidor comprometido.

5.5 Escaneo automatizado de recursos web

Durante el análisis de los registros de Nginx encontramos una actividad que nos llamó la atención. La IP 20.169.136.165 había generado 117 eventos en un periodo de tiempo bastante corto.
Al revisar las peticiones, vimos que se estaban buscando diferentes archivos PHP y directorios relacionados con WordPress, como /admin.php, /wp-admin/css/, /wp-admin/network/, /wp-content/plugins/helopress/wp_filemanager.php



Estas rutas no pertenecían a nuestra web ficticia Aegis Bank, por lo que parecía que se estaban buscando archivos o aplicaciones vulnerables, además, las peticiones se realizaron con muy poca diferencia de tiempo, algo que nos hizo sospechar que se trataba de un escaneo automatizado.
Wazuh también detectó esta actividad y generó una alerta de nivel 10, correspondiente a la regla 31151, esta regla detectaba varios errores HTTP procedentes de una misma IP.



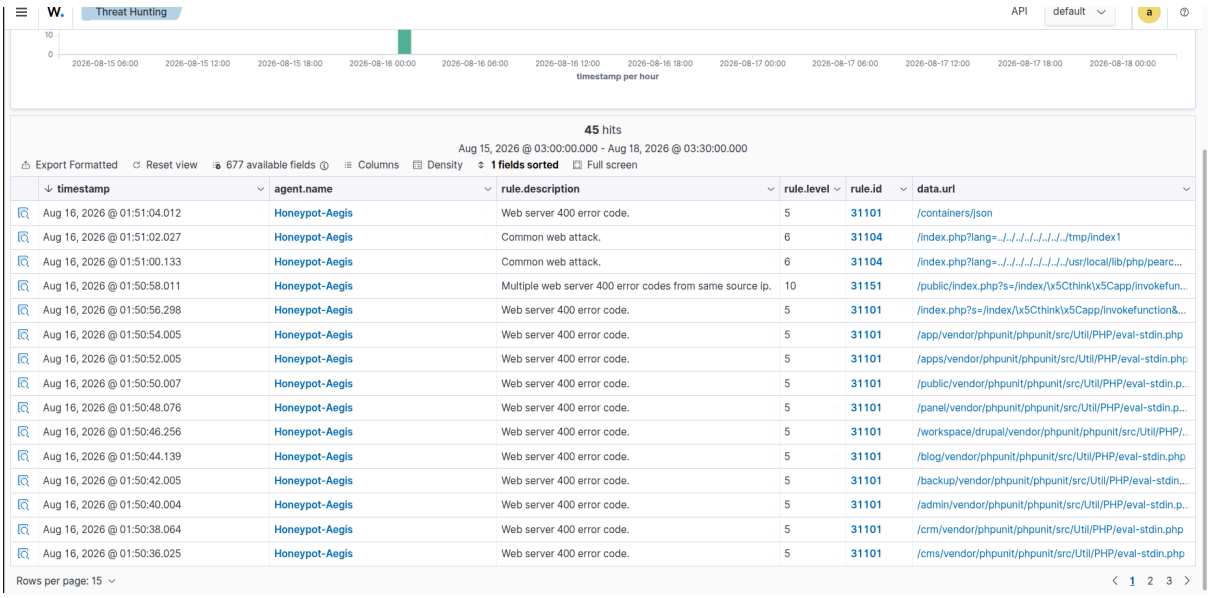
Al revisar los eventos encontramos peticiones a archivos como /new.php y /01.php, a las que el servidor había respondido con un código 404.



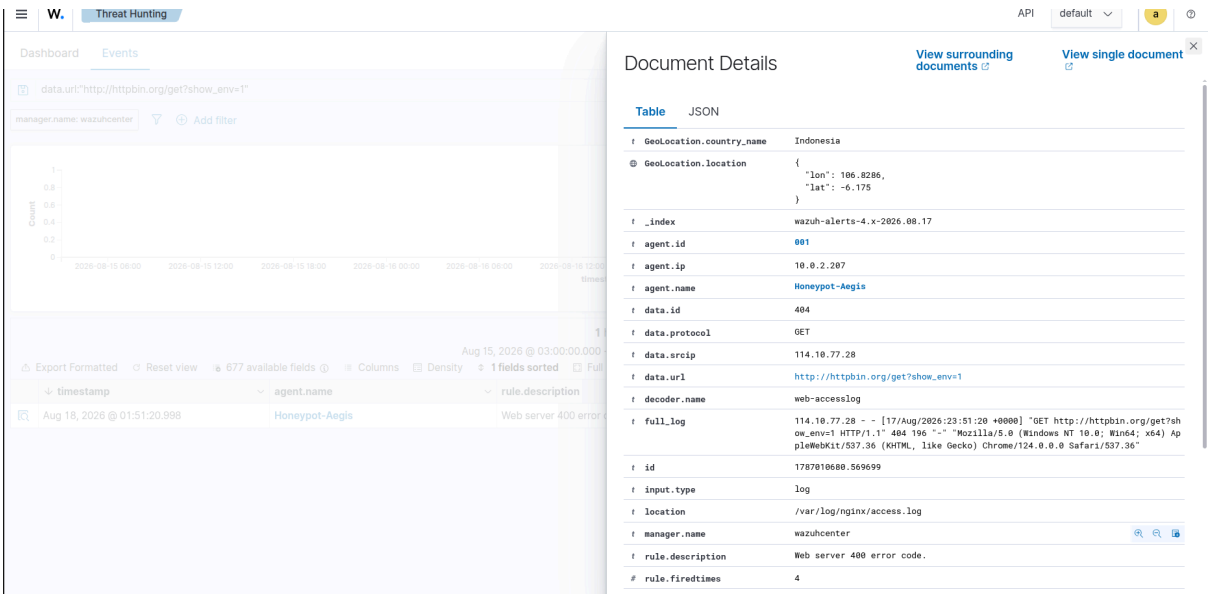
En los registros revisados tampoco aparecía información de Referer ni User-Agent, aunque esto por sí solo no demostraba que se tratara de un ataque.
Por el número de peticiones, las rutas solicitadas y la rapidez con la que se realizaron, consideramos que la actividad era compatible con un escaneo automatizado de recursos web.
Este caso nos permitió comprobar que una petición con error 404 puede no tener importancia por sí sola, pero muchas peticiones a diferentes rutas en pocos segundos pueden revelar un comportamiento sospechoso.

5.6 Análisis de intentos de explotación web

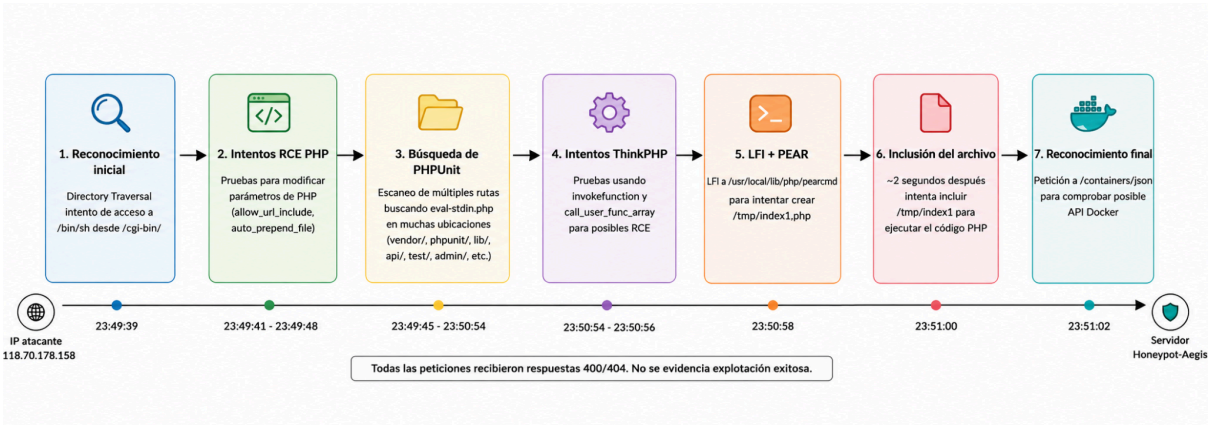
Durante el análisis de las peticiones HTTP se detectó una actividad especialmente intensa procedente de la IP 118.70.178.158.
En pocos segundos esta dirección realizó 45 peticiones diferentes, probando distintas rutas y configuraciones del servidor. Por la rapidez y el orden de las solicitudes, todo apunta a un proceso automatizado que buscaba posibles vulnerabilidades.



A diferencia de otros escaneos observados anteriormente, en este caso no se limitaba a buscar archivos o directorios. Las peticiones intentaban comprobar diferentes formas de conseguir acceso o ejecutar código en el servidor.
Uno de los casos más interesantes apareció al final de la secuencia. Primero se realizó una petición intentando utilizar pearcmd para crear un archivo llamado index1.php,



Aproximadamente dos segundos después, la misma IP realizó otra petición intentando acceder a ese archivo, es decir primero intentar crear un archivo con código y después comprobar si podía utilizarlo.
También se realizó una petición a /containers/json que podría estar relacionada con la búsqueda de un servicio Docker accesible.
Las respuestas del servidor fueron principalmente 404 y 400, por lo que no se encontró pruebas de que ninguno de estos intentos tuviera éxito.

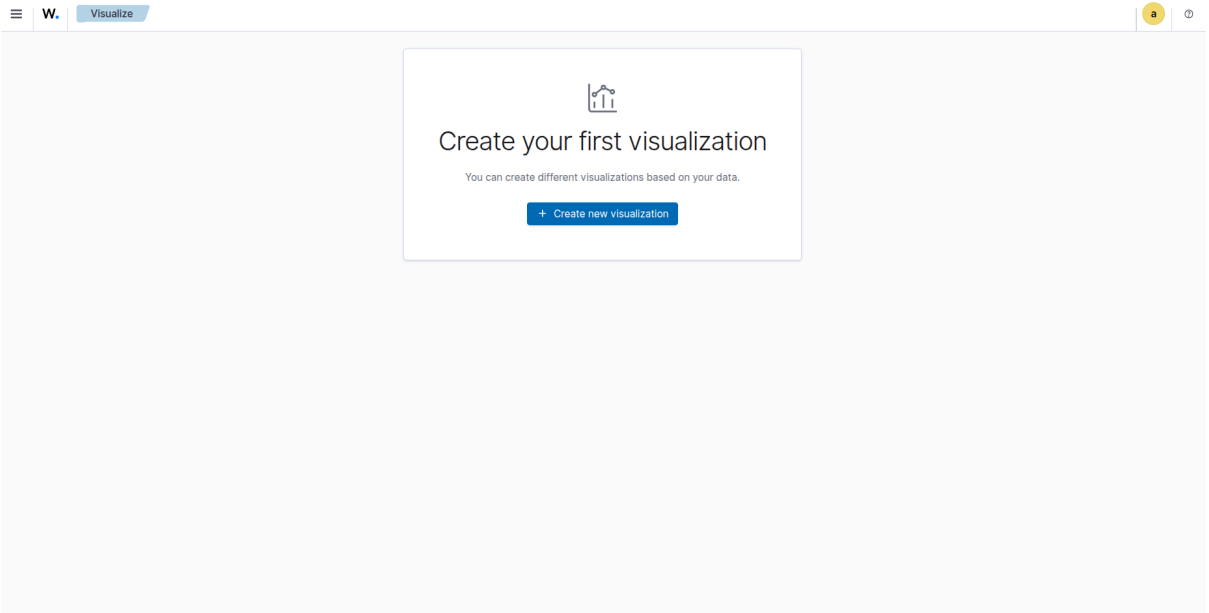


6. Creación del dashboard de monitorización

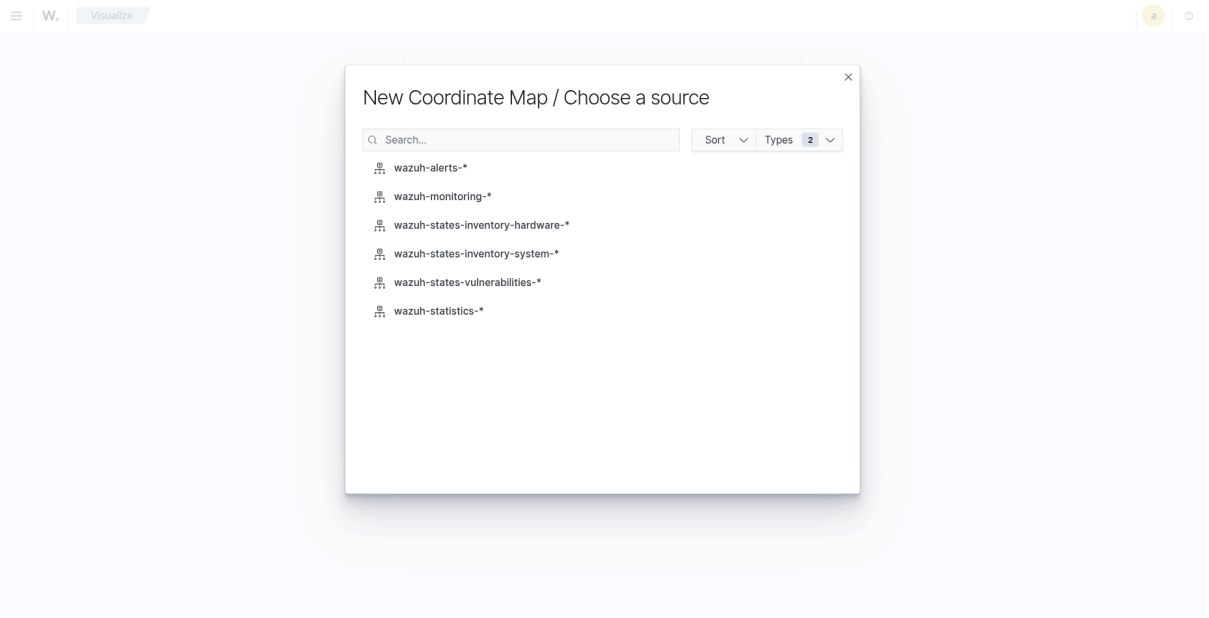
Una vez recopilada y analizada la actividad de Cowrie y Nginx, se creó un dashboard personalizado en Wazuh para visualizar de forma más clara lo que está ocurriendo en el honeypot.
La idea será reunir en una misma pantalla la información más interesante obtenida durante el periodo de monitorización: el volumen de eventos, su evolución en el tiempo, las IP que generaban más actividad, los países de origen y la actividad registrada tanto por Cowrie como por Nginx.
También se añadió un mapa mundial utilizando la información de geolocalización configurada anteriormente, lo que permitia ver de forma visual desde qué zonas se estaban recibiendo las conexiones.

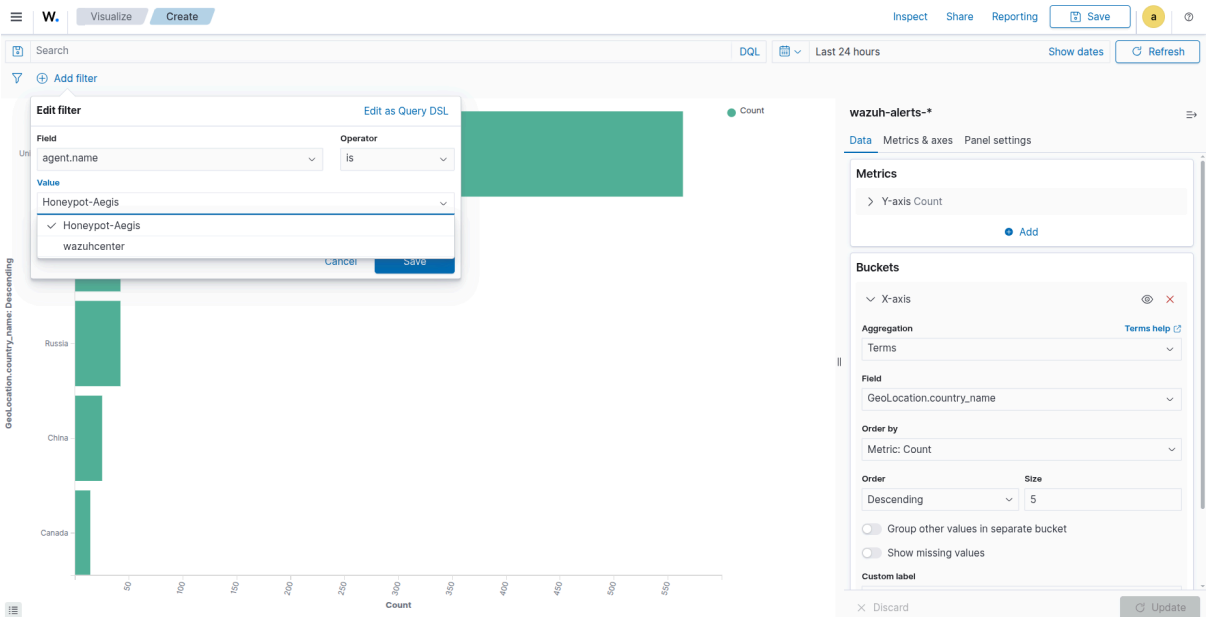
6.1 Creación del dashboard

Para reunir toda la información en una misma vista, se creó un dashboard personalizado desde Explore – Dashboards de Wazuh.



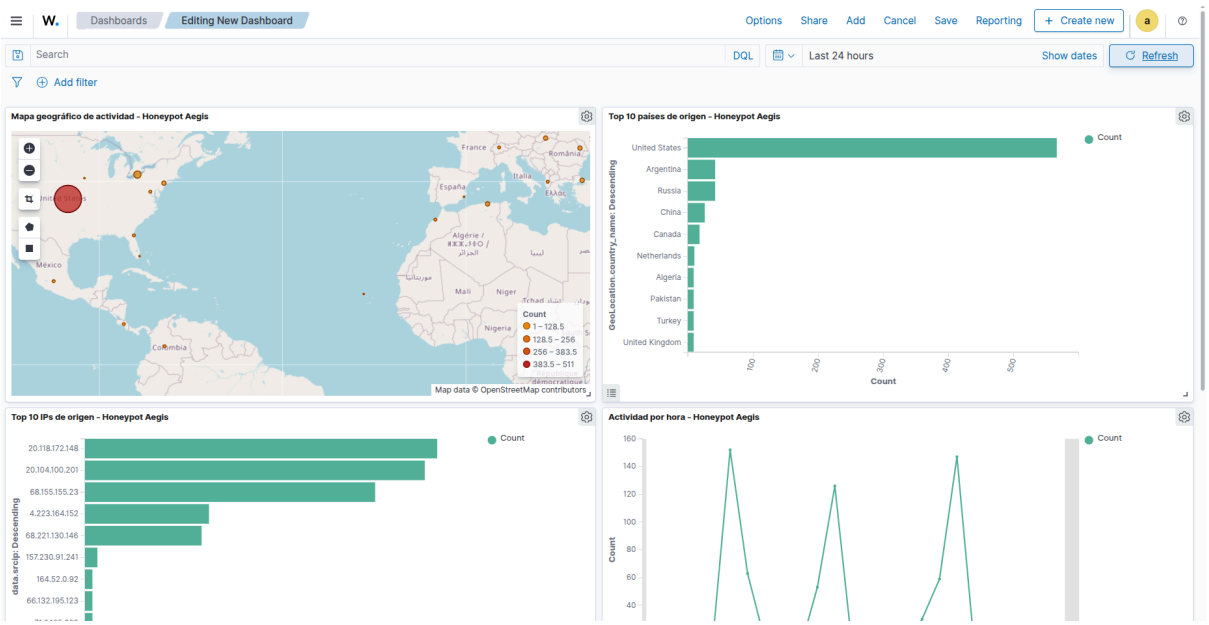
Las diferentes visualizaciones se fueron creando utilizando como fuente de datos wazuh-alerts. Además, se aplicó el siguiente filtro para mostrar únicamente los eventos correspondientes al honeypot.





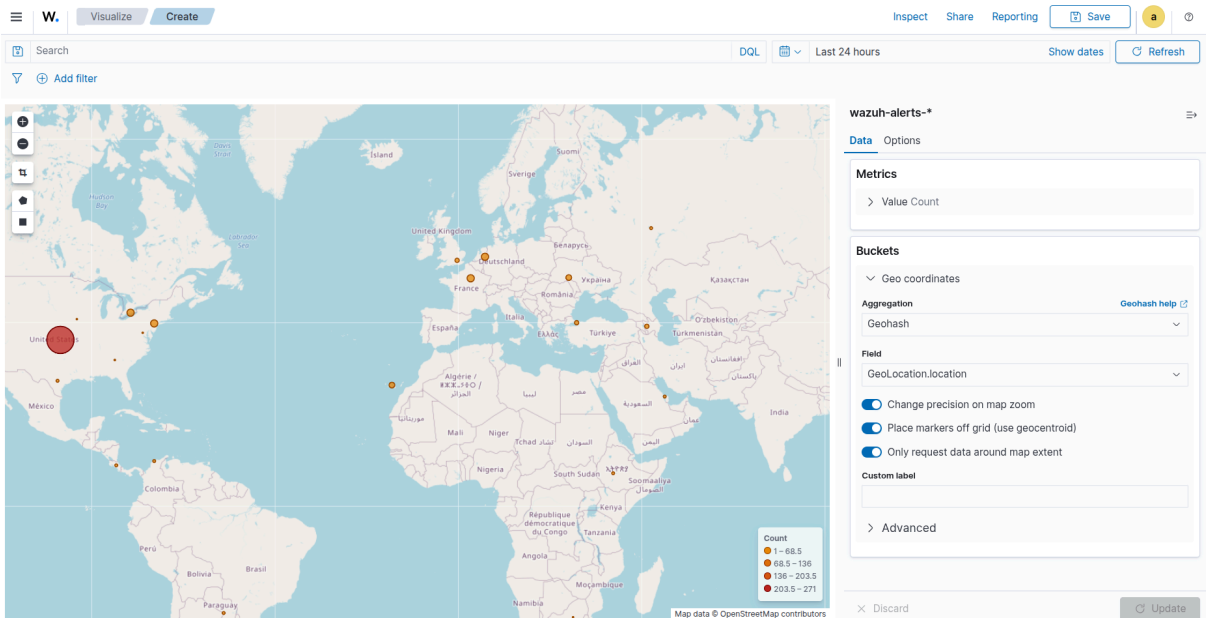
Así se evita mezclar la actividad del honeypot con eventos generados por el propio servidor Wazuh u otros agentes.

Una vez guardadas las visualizaciones, se añadieron al dashboard y se organizaron para mostrar la actividad registrada: procedencia de las conexiones, IP con mayor actividad, evolución de los eventos y datos relacionados con las sesiones SSH de Cowrie.



6.2 Mapa geográfico de actividad

Una de las visualizaciones creadas fue un mapa mundial para mostrar la procedencia aproximada de las conexiones recibidas por el honeypot.
Para crear el mapa se utilizó el campo GeoLocation.location
Este campo contiene las coordenadas asociadas a las direcciones IP públicas registradas en los eventos.

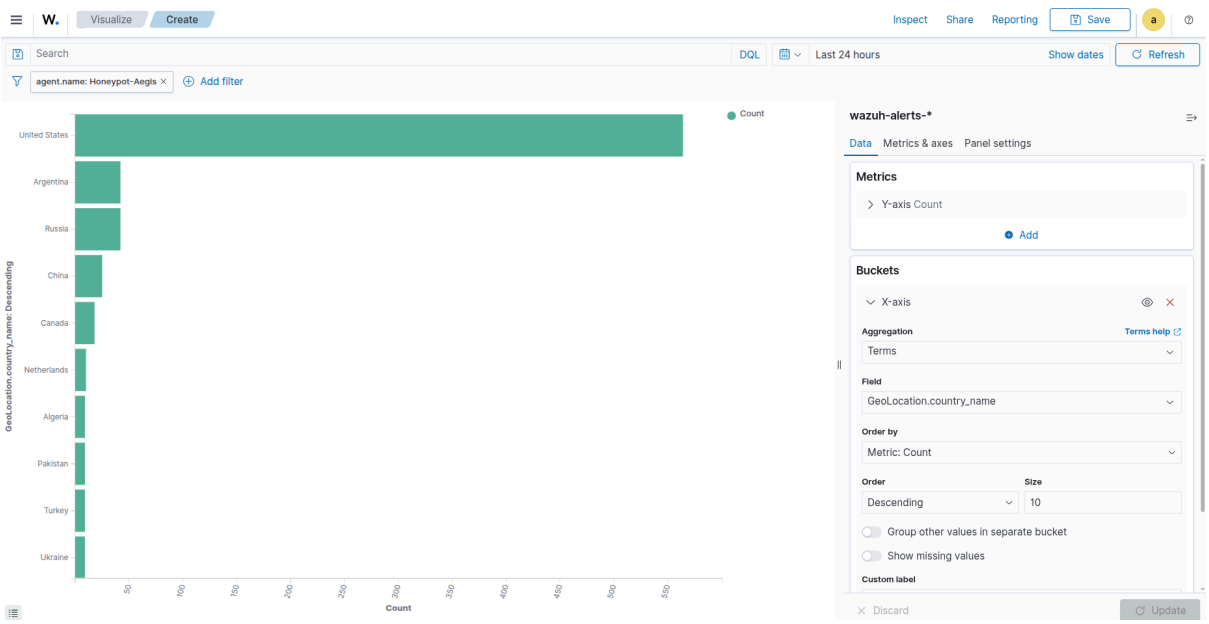


En la visualización se utilizó una agregación Geohash sobre este campo y se aplicó el filtro de Honeypot-Aegis para mostrar únicamente la actividad correspondiente al honeypot.

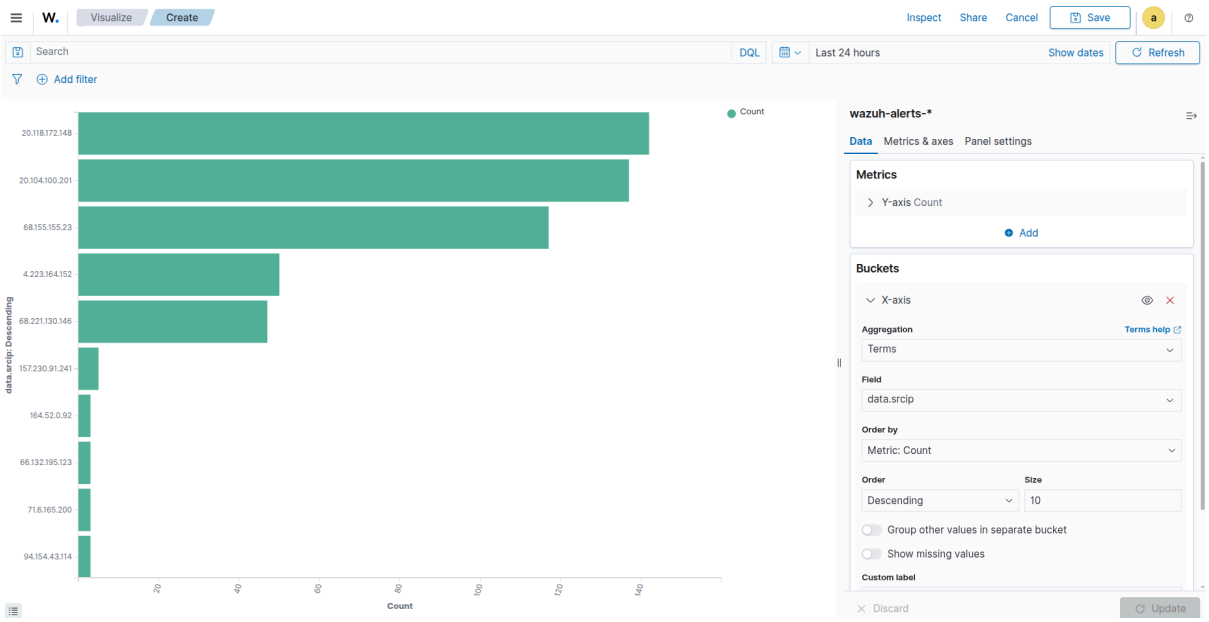
El resultado permite ver de forma rápida desde qué zonas del mundo se están recibiendo conexiones y dónde se concentra una mayor cantidad de actividad.

6.3 Principales países y direcciones IP de origen

Además del mapa, se crearon dos visualizaciones para identificar qué países y direcciones IP generaron más actividad durante el periodo de monitorización.
Top 10 países de origen que servirá para conocer los países con mayor número de eventos se utilizó el campo GeoLocation.country_name
Los resultados se agruparon para mostrar los 10 países con mayor actividad registrada por el honeypot.
Esta visualización permite comparar rápidamente la procedencia geográfica de las conexiones y comprobar qué países concentran un mayor volumen de eventos.
Hay que tener en cuenta que el número de eventos no equivale al número de ataques o atacantes, ya que una misma IP puede generar muchos registros durante una sola sesión.

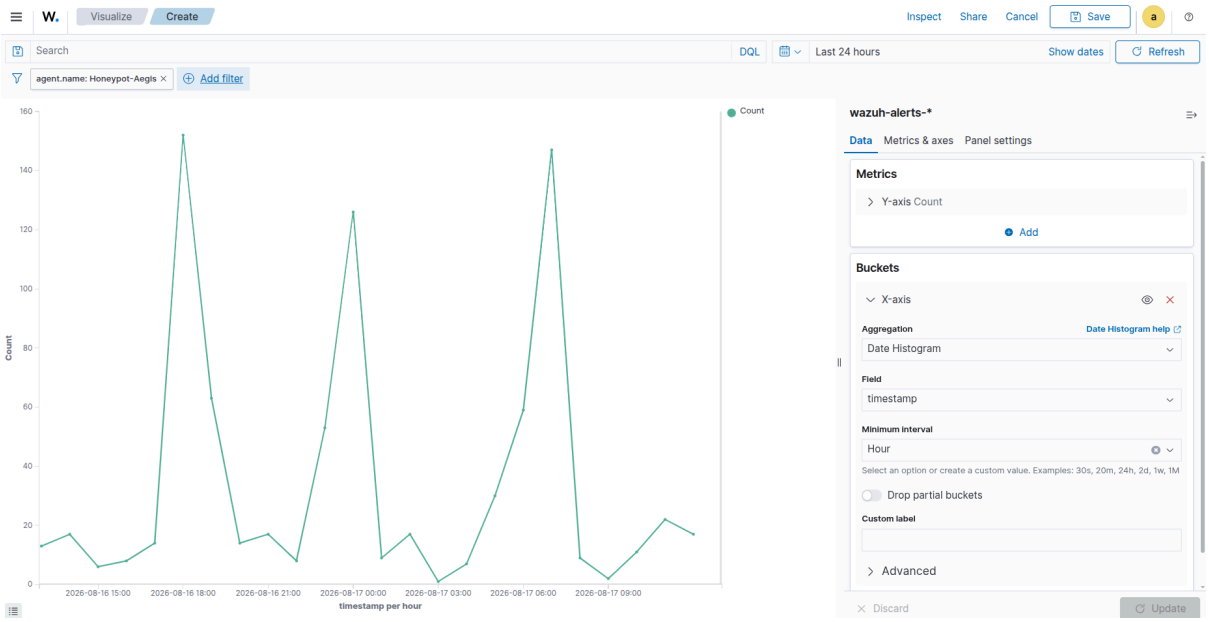


También se creó una visualización para identificar las direcciones IP que generaron más actividad, para ello se utilizó el campo data.srcip y se mostraron las diez IP con mayor número de eventos.
Este gráfico permite localizar rápidamente las direcciones más activas y, si alguna resulta especialmente interesante, utilizarla después como filtro en Threat Hunting para analizar con más detalle su comportamiento.



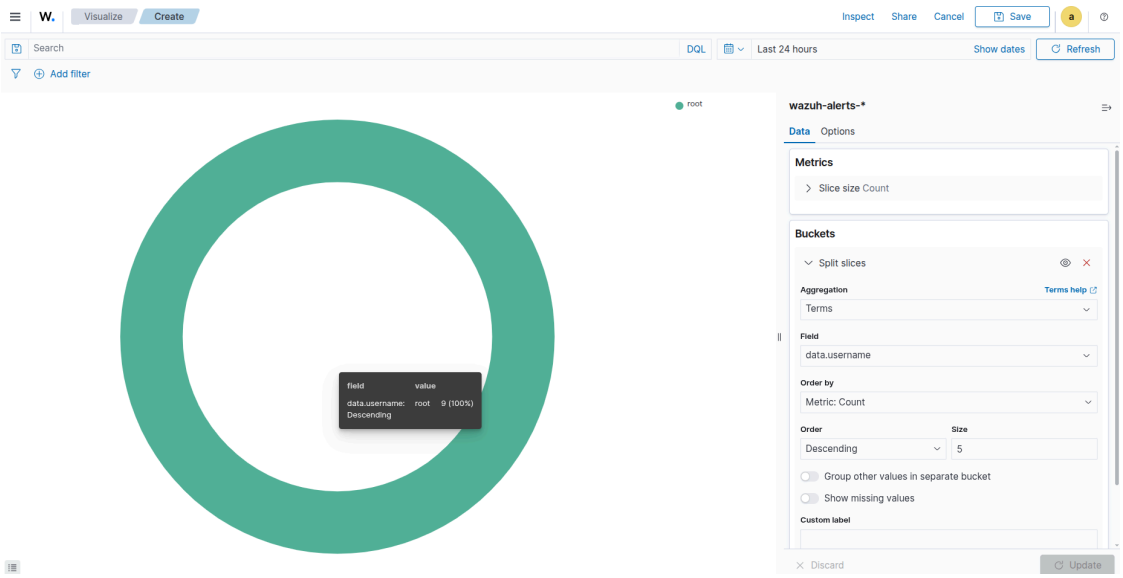
6.4 Evolución temporal de la actividad

Para ver cómo fue cambiando la actividad del honeypot a lo largo del tiempo, se creó un gráfico utilizando el campo timestamp.
Los eventos se agruparon por horas mediante Date Histogram, permitiendo observar fácilmente los momentos con mayor o menor actividad.
En el gráfico se apreciarán diferentes picos a lo largo del periodo analizado, mostrando que las conexiones no se reciben de forma constante, sino que existen franjas horarias con bastante más actividad que otras.
Esta visualización también permitirá localizar un pico concreto y después revisar en Threat Hunting qué IP o qué tipo de eventos lo generaron.



6.5 Usuarios utilizados en los intentos de acceso

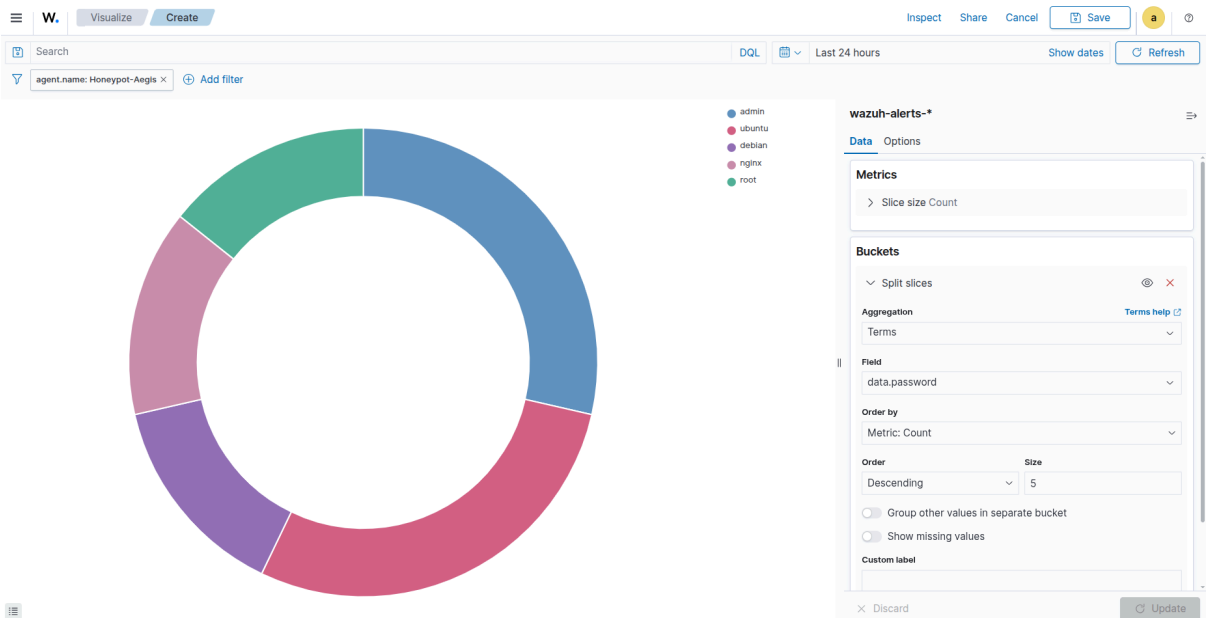
Para conocer qué nombres de usuario se probaron con más frecuencia contra Cowrie, se creó una visualización utilizando el campo data.username
Los resultados se agruparon mediante Terms y se representaron en un gráfico tipo donut, permitiendo comparar fácilmente los usuarios que aparecen con mayor frecuencia.
Durante el periodo de monitorización se observará que algunos nombres se repiten mucho más que otros, especialmente cuentas habituales en servidores Linux, como root.



Esto permite ver rápidamente qué usuarios están probando con mayor frecuencia los sistemas que intentan acceder al honeypot.

6.6 Contraseñas utilizadas en los intentos de acceso

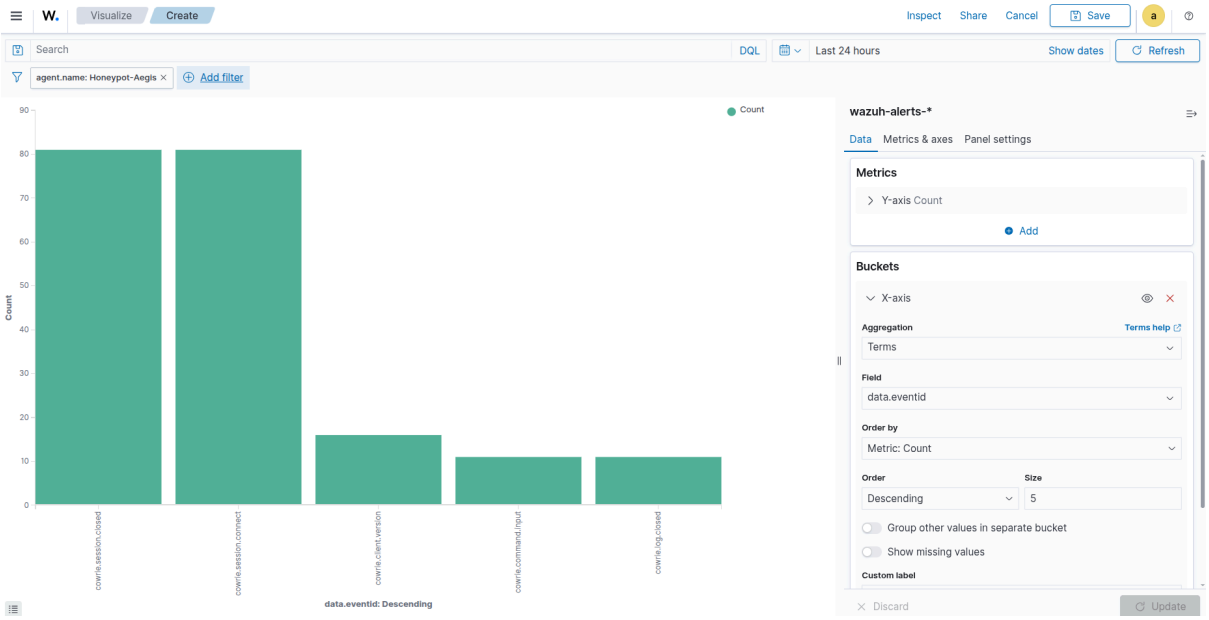
También se creó una visualización para conocer qué contraseñas se probaron con mayor frecuencia durante los intentos de autenticación contra Cowrie, para ello se utilizó el campo data.password.
Los valores se agruparon mediante Terms, mostrando las cinco contraseñas más utilizadas.
Entre los resultados aparecerán palabras sencillas y relacionadas con usuarios o servicios habituales, como admin, ubuntu, debian, nginx o root.



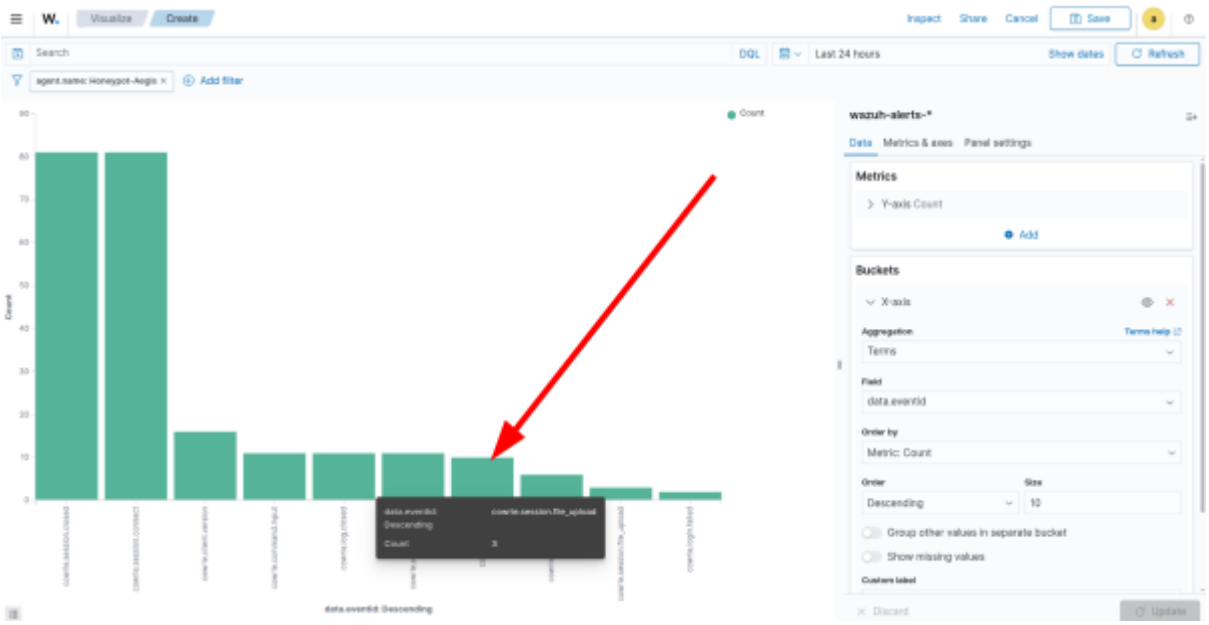
Este tipo de contraseñas hace pensar que muchas de las conexiones proceden de bots que prueban automáticamente credenciales comunes buscando servidores con accesos fáciles de adivinar.

6.7 Tipos de eventos registrados por Cowrie

Para ver qué tipo de actividad estaba registrando Cowrie, se creó una visualización utilizando el campo data.eventid.
Los eventos se agruparon por tipo, permitiendo comparar fácilmente cuáles aparecían con mayor frecuencia.



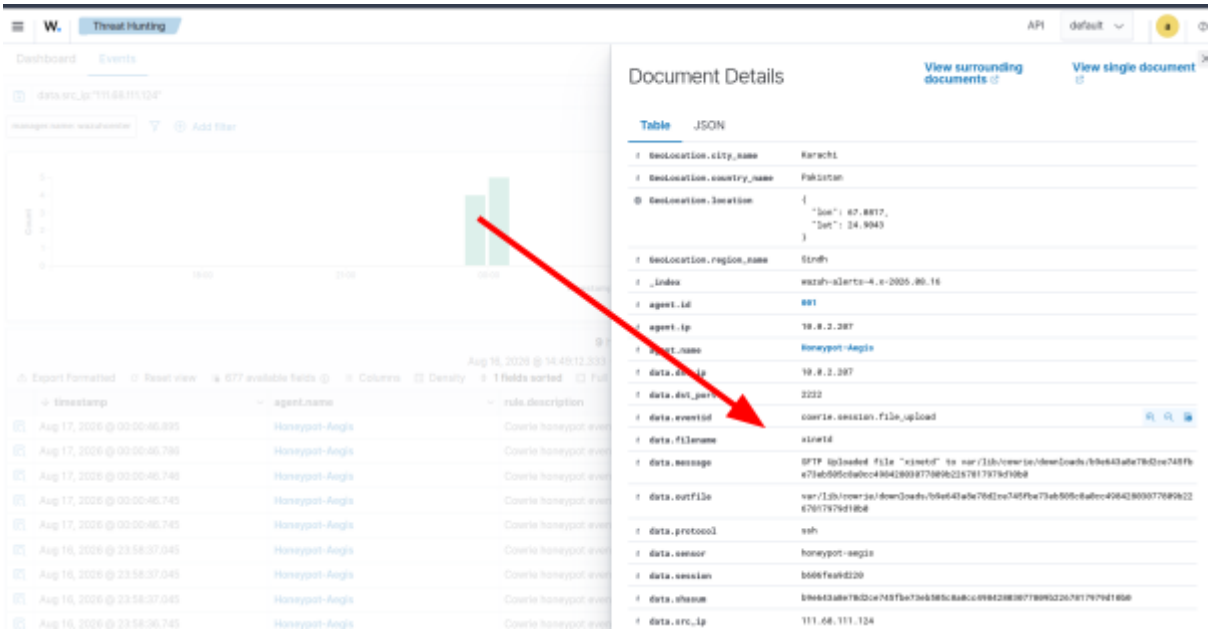
Entre ellos se encontraron conexiones, intentos de autenticación, comandos ejecutados y cierres de sesión. También aparecieron eventos especialmente interesantes relacionados con la transferencia de archivos al honeypot



La aparición de eventos cowrie.session.file_upload llamó especialmente la atención, ya que indicaba que algunos sistemas habían intentado descargar o transferir archivos dentro del entorno.
A partir de este hallazgo se analizarán los archivos capturados por Cowrie para comprobar qué contienen y si están relacionados con actividad maliciosa.

7. Análisis de archivos maliciosos capturados

Durante el análisis de los eventos de Cowrie aparecieron varios registros de tipo cowrie.session.file_upload, estos eventos indicaban que algunos sistemas habían intentado transferir archivos al entorno simulado del honeypot.
Al revisar las alertas desde Threat Hunting, se pudo consultar información como la IP de origen, el nombre del archivo, la sesión relacionada y su hash SHA-256.
Entre los archivos capturados aparecieron nombres como sshd o xinetd



Estos nombres llaman la atención porque coinciden con servicios habituales de Linux y podrían utilizarse para intentar que los archivos pasen desapercibidos. Sin embargo, el nombre por sí solo no permite saber si un archivo es malicioso, por lo que será necesario analizarlos con más detalle.

7.1 Localización de los archivos capturados

Al revisar los eventos se comprobó que Cowrie guarda una copia de los archivos transferidos dentro de su directorio de descargas
/home/cowrie/cowrie/var/lib/cowrie/downloads
Se listaron los archivos.

```
ubuntu@honeypot-aegis:~$ sudo ls -lh /home/cowrie/cowrie/var/lib/cowrie/downloads/
total 61M
-rw------- 1 cowrie cowrie 2.1M Aug 16 08:39 0ed9640f28973081f72fedf48b729f47575db8adb70dbdf8d529e1ddc43ca101
-rw------- 1 cowrie cowrie 544K Aug 16 03:35 38ef0580d99fb1524c13f8dc4981fe2757deb290b29f947ebc24b4b359756f63
-rw------- 1 cowrie cowrie 320K Aug 12 04:20 72ce5b00ca4bfa0c18fcdf03a15e5391a85d81300783626598fe7e022e0ec538
-rw------- 1 cowrie cowrie  29M Aug 12 02:54 94f2e4d8d4436874785cd14e6e6d403507b8750852f7f2040352069a75da4c00
-rw------- 1 cowrie cowrie  29M Aug 16 22:00 b9e643a8e78d2ce745fbe73eb505c8a0cc49842803077809b2267817979d10b0
-rw------- 1 cowrie cowrie    0 Aug 12 20:47 e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

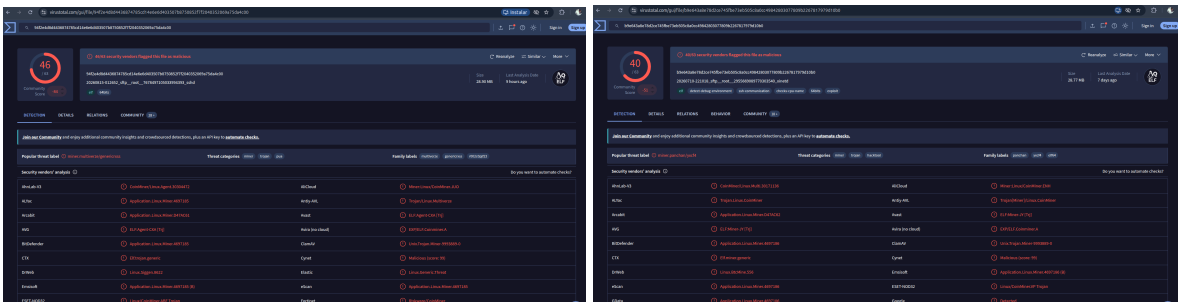
Se encontraron varios archivos de diferentes tamaños, algunos de ellos de hasta aproximadamente 29 MB. Cowrie los almacena utilizando su hash SHA-256 como nombre, en lugar de conservar únicamente el nombre original utilizado durante la transferencia.
A continuación se utilizó el comando file para identificar el tipo de archivo sin necesidad de ejecutar ninguna muestra.

```
ubuntu@honeypot-aegis:~$ sudo bash -c 'file /home/cowrie/cowrie/var/lib/cowrie/downloads/*'
/home/cowrie/cowrie/var/lib/cowrie/downloads/0ed9640f28973081f72fedf48b729f47575db8adb70dbdf8d529e1ddc43ca101: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, missing section headers at 30304408
/home/cowrie/cowrie/var/lib/cowrie/downloads/38ef0580d99fb1524c13f8dc4981fe2757deb290b29f947ebc24b4b359756f63: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, missing section headers at 30304408
/home/cowrie/cowrie/var/lib/cowrie/downloads/72ce5b00ca4bfa0c18fcdf03a15e5391a85d81300783626598fe7e022e0ec538: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, missing section headers at 30304408
/home/cowrie/cowrie/var/lib/cowrie/downloads/94f2e4d8d4436874785cd14e6e6d403507b8750852f7f2040352069a75da4c00: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=300bf5c7e304c732122f0b0fa290bda984441bd6, for GNU/Linux 3.2.0, stripped
/home/cowrie/cowrie/var/lib/cowrie/downloads/b9e643a8e78d2ce745fbe73eb505c8a0cc49842803077809b2267817979d10b0: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, Go BuildID=OttNCKJJuAQ4NQ-prOfa/SXOw9b5r_XbrMIXoAQre/t5Z4BcUk-VUDnL_-FoBx/RAVMiaQJu_NAEOO6wzkN, stripped
/home/cowrie/cowrie/var/lib/cowrie/downloads/e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855: empty
```

El resultado mostró que cinco de las muestras eran ejecutables ELF de 64 bits para Linux, mientras que otro archivo estaba vacío, en uno de los ejecutables también se identificaron referencias que indicaban que había sido compilado utilizando Go.
En ningún momento se ejecutaron los archivos, sino que, el análisis se realizará utilizando solo los hashes y otra información estática de las muestras.

7.2 Análisis de las muestras mediante SHA-256

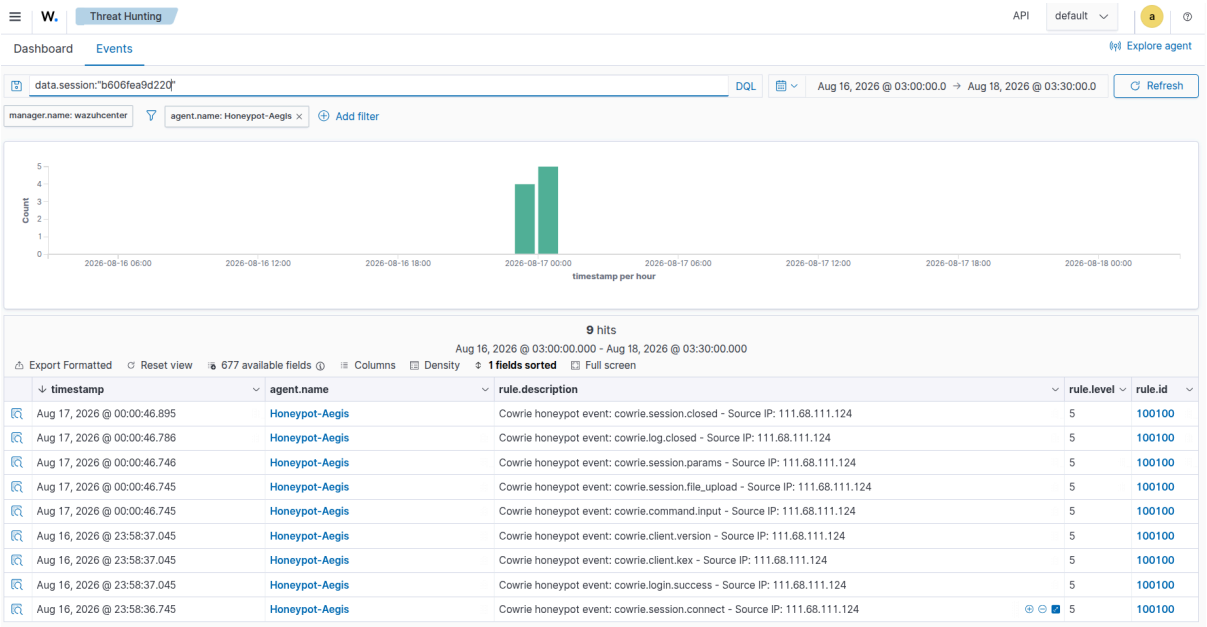
Una vez identificados los archivos como ejecutables ELF para Linux, se comprobó si las muestras ya habían sido detectadas como maliciosas.
Para realizar el análisis de forma segura no se ejecutaron ni se subieron los archivos. Se utilizaron sus hashes SHA-256, que además coincidían con los nombres utilizados por Cowrie para almacenarlos.
Los hashes se consultaron en VirusTotal, donde se comprobó que los cinco ejecutables encontrados ya eran detectados como maliciosos por numerosos motores antivirus.



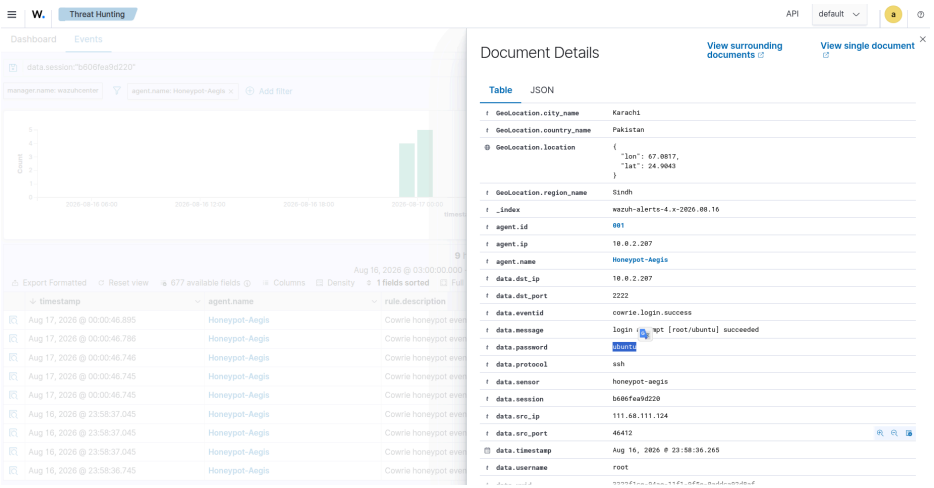
Las clasificaciones estaban relacionadas principalmente con troyanos y malware de minería de criptomonedas para sistemas Linux. Entre los nombres detectados aparecieron familias o clasificaciones como CoinMiner, Multiverze y Panchan.
Estos resultados confirmaron que el honeypot había llegado a capturar muestras reales de malware enviadas por sistemas que encontraron el servicio SSH expuesto en Internet.
Las muestras fueron almacenadas por el honeypot para su análisis y en ningún momento se ejecutaron en el servidor.

7.3 Reconstrucción de un intento de infección

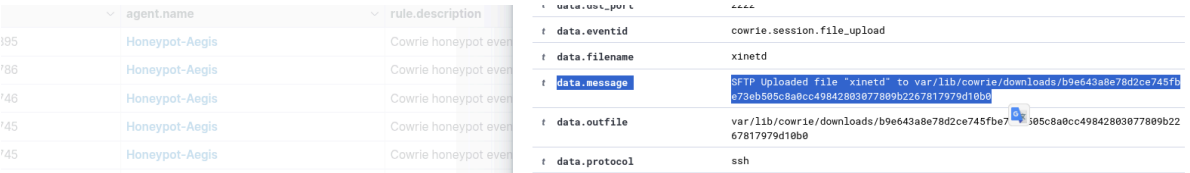
Una vez confirmado que algunos de los archivos capturados eran malware, se revisaron las sesiones relacionadas con estas transferencias para ver qué había ocurrido antes y después de la descarga.
Uno de los casos más interesantes procedía de la IP 111.68.111.124 así que, utilizando el identificador data.session fue posible agrupar los eventos de una misma conexión y reconstruir la toda la actividad.



La sesión comenzó con un evento cowrie.login.success. Se utilizaron las siguientes credenciales root:ubuntu



Después del acceso se registró un evento cowrie.session.file_upload, el sistema remoto transfirió mediante SFTP un archivo llamado xinetd.



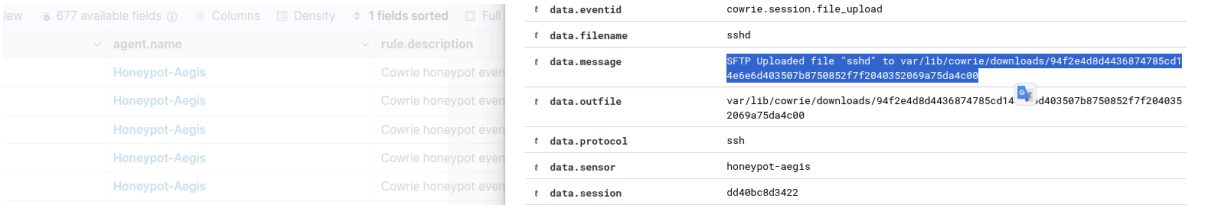
Poco después de transferir el archivo el atacante, intenta dar permisos de ejecución al archivo y nohup ... & intenta ejecutarlo en segundo plano para que continúe funcionando aunque termine la sesión.

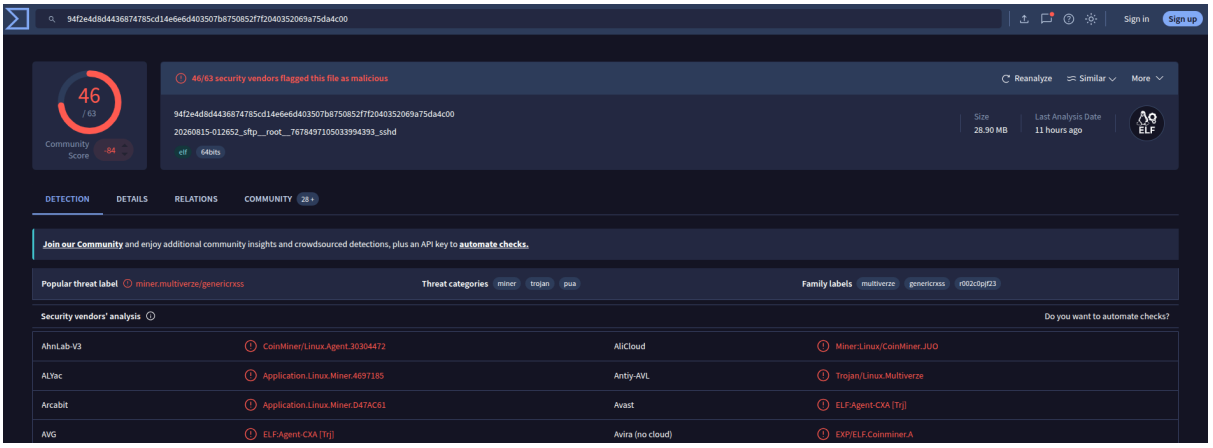


La rapidez con la que se realizaron las diferentes acciones y el uso de una secuencia de comandos ya preparada hacen pensar que probablemente se trataba de un proceso automatizado.

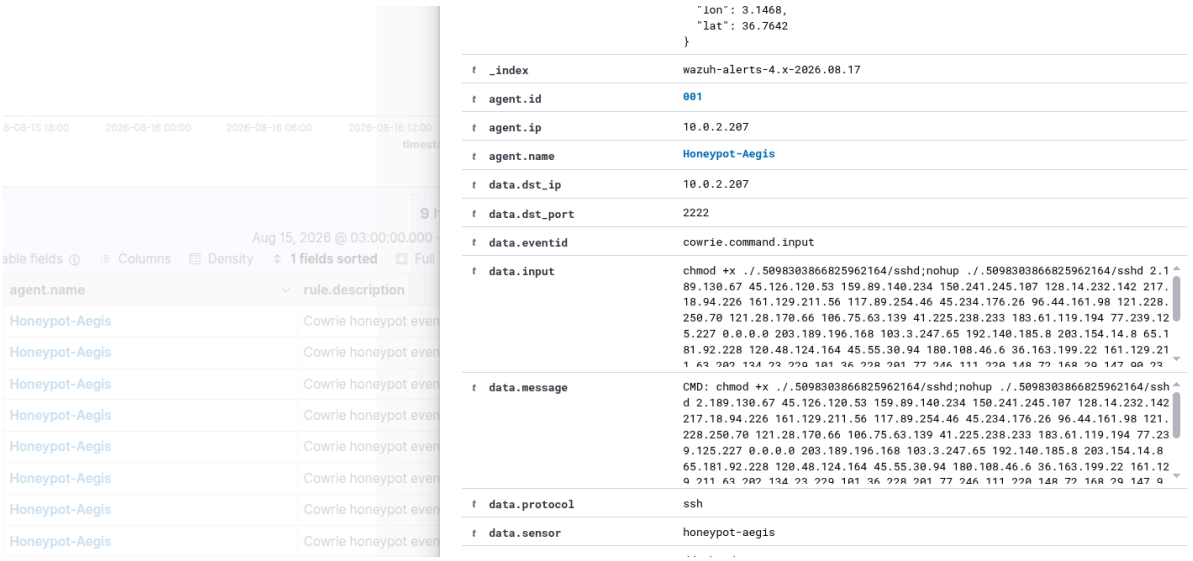
7.4 Análisis de un segundo intento de infección

Durante el análisis se encontró una segunda sesión en la que también se transfirió un ejecutable al honeypot. En este caso, la conexión procedía de la IP 41.111.172.25
Cowrie registró mediante SFTP la transferencia de un archivo llamado sshd, también otro nombre que coincide con el servicio habitual SSH de linux.
Cowrie guardó la muestra y calculó su hash SHA-256, al consultar en VirusTotal, la muestra presentaba 46 detecciones de 63 motores, con clasificaciones relacionadas principalmente con Miner, Trojan y CoinMiner.



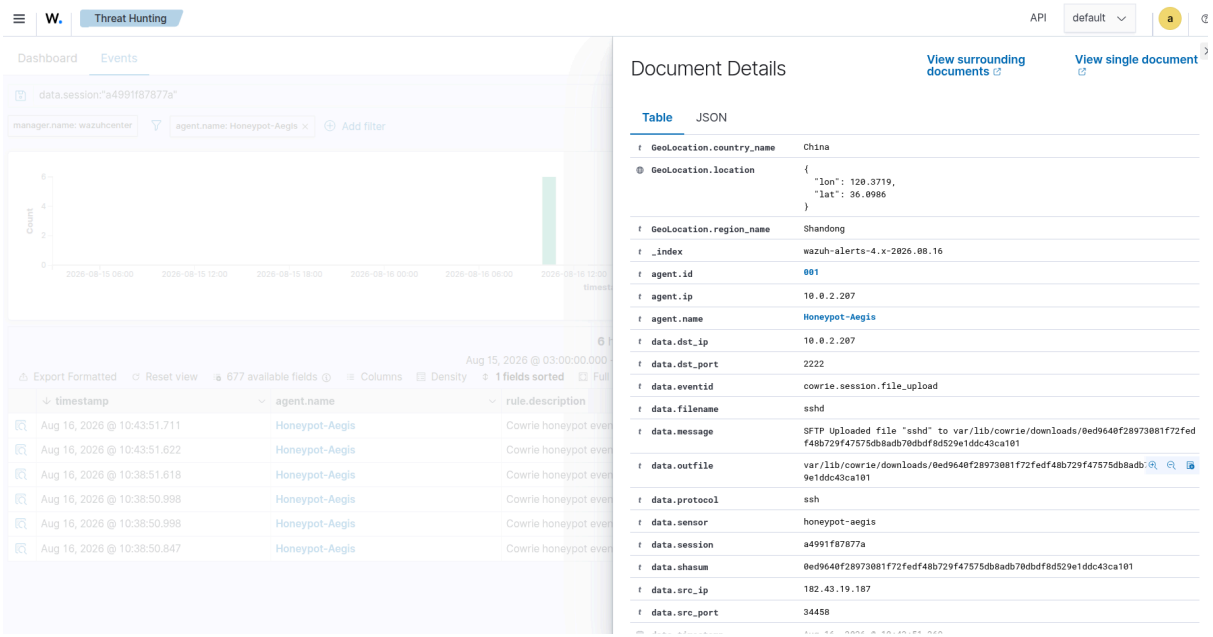


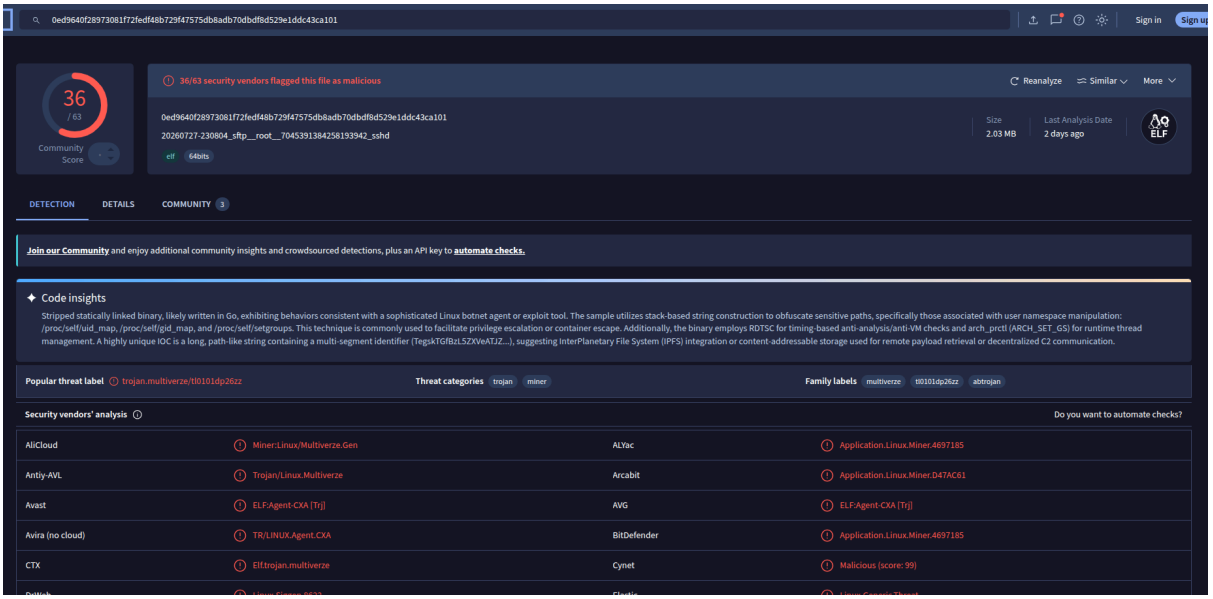
Al revisar el resto de la sesión se encontró también el intento de ejecución del archivo , lo más llamativo de este caso es que el malware no se intentó ejecutar solo. Junto al comando aparecía una larga lista de direcciones IP, que se pasaban al programa en el momento de iniciarlo.
Esto podria indicar que el ejecutable probablemente estaba preparado para utilizar esas direcciones IP de alguna forma, por ejemplo para buscar otros sistemas accesibles, realizar conexiones o intentar propagarse, aunque no podemos saber con certeza que es lo que iba ha hacer con esas IP.



7.5 Análisis de muestras adicionales

Durante el periodo de monitorización continuaron apareciendo nuevas transferencias de archivos hacia Cowrie. En varias sesiones se observó un patrón parecido: después de acceder al honeypot, se transfería mediante SFTP un archivo llamado sshd.
Una de esas muestras venia de la IP 182.43.19.187 y tenía el hash 0ed9640f28973081f72fedf48b729f47575db8adb70dbdf8d529e1ddc43ca101
Al consultar el hash en VirusTotal, 36 de 63 motores identificaban la muestra como maliciosa, principalmente como troyano y malware de minería, apareciendo también referencias a Multiverze.
Lo interesante es que esta muestra fue enviada desde una IP diferente a las analizadas anteriormente. Esto podría decir que existe algún tipo de actividad automatizada relacionada, aunque con la información disponible no era posible asegurar que todas las muestras formasen parte de una misma campaña.
En esta sesión, además, no se registraron comandos después de la transferencia. Por tanto, podemos confirmar que el archivo fue enviado al honeypot, pero no que se intentó ejecutar





Conclusiones

Con este proyecto se ha conseguido crear un honeypot en AWS y ver de forma real qué ocurre cuando dejamos un servidor expuesto a Internet. En poco tiempo empezaron a aparecer conexiones SSH, intentos de acceso y escaneos buscando todo tipo de archivos y rutas en la web.

Gracias a Wazuh se ha podido recoger toda esta actividad y analizarla desde un mismo lugar. También se ha podido ver de dónde procedían las conexiones, seguir algunas sesiones de Cowrie y observar los comandos que se ejecutaban después de conseguir entrar en el sistema simulado.

Durante el proyecto también fueron apareciendo algunos problemas que obligaron a cambiar parte de la configuración inicial. Resolverlos ha servido para entender mucho mejor cómo se comunican AWS, WireGuard, VirtualBox y Wazuh. Además, se ha podido aprovechar parte de la infraestructura que ya teníamos del proyecto anterior en lugar de empezar desde cero.

Aunque el proyecto termina aquí, todavía se puede ampliar bastante. Como siguiente paso sería interesante mejorar el honeypot web añadiendo nuevos señuelos, como un panel de administración o un archivo .env falso, y ver qué hacen los sistemas que los encuentran. También se podría crear una parte más interactiva con Flask y automatizar la actualización del DNS cuando cambie la IP pública de AWS.